

Week 08 — Quantities of Interest & OLS Assumptions

Expected values, first differences and predicted values by bootstrap, and what happens to OLS when its assumptions fail: Monte Carlo simulations

AUTHORS

Johann-Friedrich Salzmann

Joshua Elias Schmidt Rodrigues

DATE

2026-10-29

Topics:

Package management . . . 2
Quantities of interest . . . 2
From the bootstrap to Monte Carlo . . . 23
The classical linear model assumptions . . . 23
Monte Carlo simulation . . . 24
The population . . . 24
No violation: the sampling distribution . . . 29
Omitted variable . . . 34
Measurement error . . . 35
Heteroskedasticity . . . 39
Multicollinearity . . . 44
All rounds compared . . . 48
Concluding remarks . . . 51
Try it: survey size . . . 52
Try it: stronger measurement error . . . 52
Try it: curved education effect . . . 53
Try it: clustered surveys . . . 55
Try it: skewed errors . . . 57

Package management

Nothing new this week: `marginaleffects` and `infer` from Week 7 for the quantities of interest, `infer` again for a bootstrap inside the simulation, and `ggdag` and `dagitty` from Week 5 for the DAG.

```
if (!require(pacman)) install.packages("pacman")
```

MESSAGE

```
Loading required package: pacman
```

```
p_load(tidyverse, broom, marginaleffects, infer, ggdag, dagitty, patchwork)
set.seed(808)
```

Quantities of interest

We pick up Week 7's age model to answer substantive questions. The lecture's **quantities of interest** (estimands) are functions of the coefficients:

- **Marginal effect** $\frac{\partial Y}{\partial X}$: how Y changes with X, all else equal, as in Week 7.
- **Expected value** $E(Y \mid X)$: the average Y for given values of the predictors.
- **First difference** $E(Y \mid X_1) - E(Y \mid X_2)$: the difference between two expected values. Elsewhere it is called a *predictive comparison* (Gelman & Hill) or a *contrast*; `marginaleffects` calls it a *comparison* (`comparisons()`).
- **Predicted value** $\hat{Y} \mid X$: a single new observation for given values of the predictors.

The counties, the model and the values we picked, as in Week 7:

```
gerda <- read_csv("data/federal_cty_harm.csv")
```

MESSAGE

```
Rows: 4000 Columns: 127
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr    (2): county_code, state
dbl  (124): election_year, eligible_voters, number_voters, valid_votes, inva...
date   (1): election_date

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
covariates <- read_csv("data/cty_covariates.csv")
```

MESSAGE

```
Rows: 11200 Columns: 32
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr  (1): county_code
dbl (31): year, share_65plus, median_age, share_foreign, share_female, gdp_p...

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
covariates_2021 <- covariates |>
  filter(year == 2021) |>
  select(county_code, median_age)

counties <- gerda |>
  filter(
    election_year == 2021,
    state != "11"
  ) |>
  select(county_code, state, afd, turnout) |>
  mutate(across(c(afd, turnout), ~ .x * 100)) |>
  left_join(covariates_2021, by = "county_code")
```

$$\text{AfD}_i = \beta_0 + \beta_1 \text{Turnout}_i + \beta_2 \text{Age}_i + \beta_3 \text{Turnout}_i \cdot \text{Age}_i + \varepsilon_i$$

```r
age_model <- lm(afd ~ turnout * median_age, data = counties)

age_beta_1 <- age_model |>
  tidy() |>
  filter(term == "turnout") |>
  pull(estimate)
age_beta_3 <- age_model |>
  tidy() |>
  filter(term == "turnout:median_age") |>
  pull(estimate)
```

```r
age_levels <- counties |>
  summarize(
    p10 = quantile(median_age, 0.1),
    p90 = quantile(median_age, 0.9)
  ) |>
  pivot_longer(
    everything(),
    names_to        = "percentile",
    names_prefix    = "p",
    names_transform = as.numeric,
    values_to       = "median_age"
  )

turnout_min <- counties |>
  summarize(min(turnout)) |>
  pull()
turnout_max <- counties |>
  summarize(max(turnout)) |>
  pull()
turnout_grid <- seq(turnout_min, turnout_max, length.out = 50)
```

Bootstrapping the coefficients

The lecture fits the model once and then draws coefficients from the multivariate Normal $\mathcal{MVN}\left(\hat{\beta}, \hat{\boldsymbol{V}}\right)$. We take the bootstrap from Week 3 instead: draw 399 counties with replacement, fit the model again, repeat. No Normal distribution and no $\hat{\boldsymbol{V}}$ are needed. The steps:

1. Draw bootstrap samples: 399 counties with replacement, many times.
2. Fit the model on each sample. The coefficients form their sampling distribution.
3. Choose covariate values, a **scenario**.
4. Calculate the quantity of interest (estimand) with each bootstrap sample's coefficients.
5. Summarize its distribution.

Step 1. As in Week 3, `expand_grid()` builds one block of rows per bootstrap sample, numbered by `sample_id`, here the whole `counties` tibble each time. `slice_sample()` then draws the counties of each sample with replacement:

```r
n_boot <- 1000

boot_samples <- expand_grid(
  sample_id = 1:n_boot,
  counties
) |>
  group_by(sample_id) |>
  slice_sample(prop = 1, replace = TRUE) |>
  ungroup()
boot_samples
```

OUTPUT

```
# A tibble: 399,000 × 6
   sample_id county_code state   afd turnout median_age
       <int> <chr>       <chr> <dbl>   <dbl>      <dbl>
 1         1 09181       09     7.37    83.7       43.8
 2         1 16074       16    24.5     78.4       47.7
 3         1 09190       09     8.03    82.6       44.4
 4         1 15082       15    22.1     65.9       49
 5         1 08236       08    12.5     79.7       44.2
 6         1 09181       09     7.37    83.7       43.8
 7         1 13072       13    18.0     73.2       46.9
 8         1 06631       06    13.3     78.6       43.9
 9         1 09263       09    11.6     67.6       44.3
10         1 09673       09    10.1     81.3       45.2
# ℹ 398,990 more rows
```

Step 2. We fit the model on each bootstrap sample. This gives the bootstrap coefficients:

$$\tilde{\beta}^{(r)} = \text{OLS estimate on bootstrap sample } r, \quad r = 1, ..., R$$

`rowwise()` works through one replicate r per row: filter its sample (`sample_id == replicate`), fit and tidy. Within `rowwise()`, each row's result must be a single row, so `pivot_wider()` puts the four coefficients side by side; the `mutate()` argument has no name, so they become ordinary columns. `rename()` gives the intercept the name `infer` uses below, and `pivot_longer()` at the end gives one row per replicate and coefficient:

```r
boot_coefs <- tibble(replicate = 1:n_boot) |>
  rowwise() |>
  mutate(
    boot_samples |>
      filter(sample_id == replicate) |>
      lm(afd ~ turnout * median_age, data = _) |>
      tidy() |>
      select(term, estimate) |>
      pivot_wider(names_from = term, values_from = estimate)
  ) |>
  ungroup() |>
  rename(intercept = `(Intercept)`) |>
  pivot_longer(-replicate, names_to = "term", values_to = "estimate")
boot_coefs
```

OUTPUT

```
# A tibble: 4,000 × 3
   replicate term                estimate
       <int> <chr>                  <dbl>
 1         1 intercept          -338.
 2         1 turnout               3.44
 3         1 median_age            8.25
 4         1 turnout:median_age   -0.0826
 5         2 intercept          -272.
 6         2 turnout               2.41
 7         2 median_age            6.69
 8         2 turnout:median_age   -0.0587
 9         3 intercept          -365.
10         3 turnout               3.88
# ℹ 3,990 more rows
```

The same in one compressed pipe with `infer`: `generate()` shuffled in Week 6, with `type = "bootstrap"` it resamples, and `fit()` estimates the model once per replicate. It returns the same long shape as `boot_coefs`, whose names we chose to match: `replicate`, and `intercept` for the intercept.

```
boot_coefs_infer <- counties |>
  specify(afd ~ turnout * median_age) |>
  generate(reps = n_boot, type = "bootstrap") |>
  fit()
boot_coefs_infer
```

OUTPUT

```
# A tibble: 4,000 × 3
# Groups:   replicate [1,000]
   replicate term                estimate
       <int> <chr>                  <dbl>
 1         1 intercept          -183.
 2         1 turnout               1.47
 3         1 median_age            4.88
 4         1 turnout:median_age   -0.0398
 5         2 intercept          -321.
 6         2 turnout               3.38
 7         2 median_age            7.84
 8         2 turnout:median_age   -0.0809
 9         3 intercept          -162.
10         3 turnout               1.26
# ℹ 3,990 more rows
```

`marginaleffects` bootstraps the coefficients too: `hypotheses()` returns them, `inferences()` resamples, and `get_draws()` gives one row per replicate and coefficient. We bring it into the same shape:

```
boot_coefs_marginaleffects <- hypotheses(age_model) |>
  inferences(method = "boot", R = n_boot) |>
  get_draws() |>
  as_tibble() |>
  select(replicate = drawid, term, estimate = draw) |>
  mutate(term = if_else(term == "(Intercept)", "intercept", term))
boot_coefs_marginaleffects
```

OUTPUT

```
# A tibble: 4,000 × 3
   replicate term                estimate
   <fct>     <chr>                  <dbl>
 1 1         intercept          -341.
 2 1         turnout               3.62
 3 1         median_age            8.35
 4 1         turnout:median_age   -0.0872
 5 2         intercept          -111.
 6 2         turnout               0.462
 7 2         median_age            3.25
 8 2         turnout:median_age   -0.0170
 9 3         intercept          -268.
10 3         turnout               2.57
# ℹ 3,990 more rows
```

The standard deviation of each coefficient across replicates is its bootstrap standard error:

$$\mathrm{SE}_{\mathrm{boot}(\hat{\beta}_j)} = \mathrm{sd}\left(\tilde{\beta}_j^{(1)}, \ldots, \tilde{\beta}_j^{(R)}\right)$$

```
boot_se <- boot_coefs |>
  group_by(term) |>
  summarize(se_boot = sd(estimate))

infer_se <- boot_coefs_infer |>
  group_by(term) |>
  summarize(se_infer = sd(estimate))

marginaleffects_se <- boot_coefs_marginaleffects |>
  group_by(term) |>
  summarize(se_marginaleffects = sd(estimate))

age_se <- age_model |>
  tidy() |>
  select(term, se_ols = std.error) |>
  mutate(term = if_else(term == "(Intercept)", "intercept", term))

age_se |>
  left_join(boot_se, by = "term") |>
  left_join(infer_se, by = "term") |>
  left_join(marginaleffects_se, by = "term")
```

OUTPUT

```
# A tibble: 4 × 5
  term                se_ols se_boot se_infer se_marginaleffects
  <chr>                <dbl>   <dbl>    <dbl>              <dbl>
1 intercept          78.4    80.3     77.6               78.3
2 turnout             1.05    1.08     1.05               1.05
3 median_age          1.74    1.81     1.74               1.76
4 turnout:median_age  0.0233  0.0243   0.0236             0.0237
```

All three bootstraps agree, up to the randomness of the draws. Our own standard errors are 2 to 4% larger than the OLS ones. The OLS formula assumes that all counties scatter equally around the regression line. The bootstrap only uses the counties themselves: where some scatter more than others, its standard errors come out larger.

From here on we work with one row per replicate and one column per coefficient:

```r
boot_betas <- boot_coefs |>
  pivot_wider(names_from = term, values_from = estimate) |>
  rename(
    beta_0 = intercept,
    beta_1 = turnout,
    beta_2 = median_age,
    beta_3 = `turnout:median_age`
  )
boot_betas
```

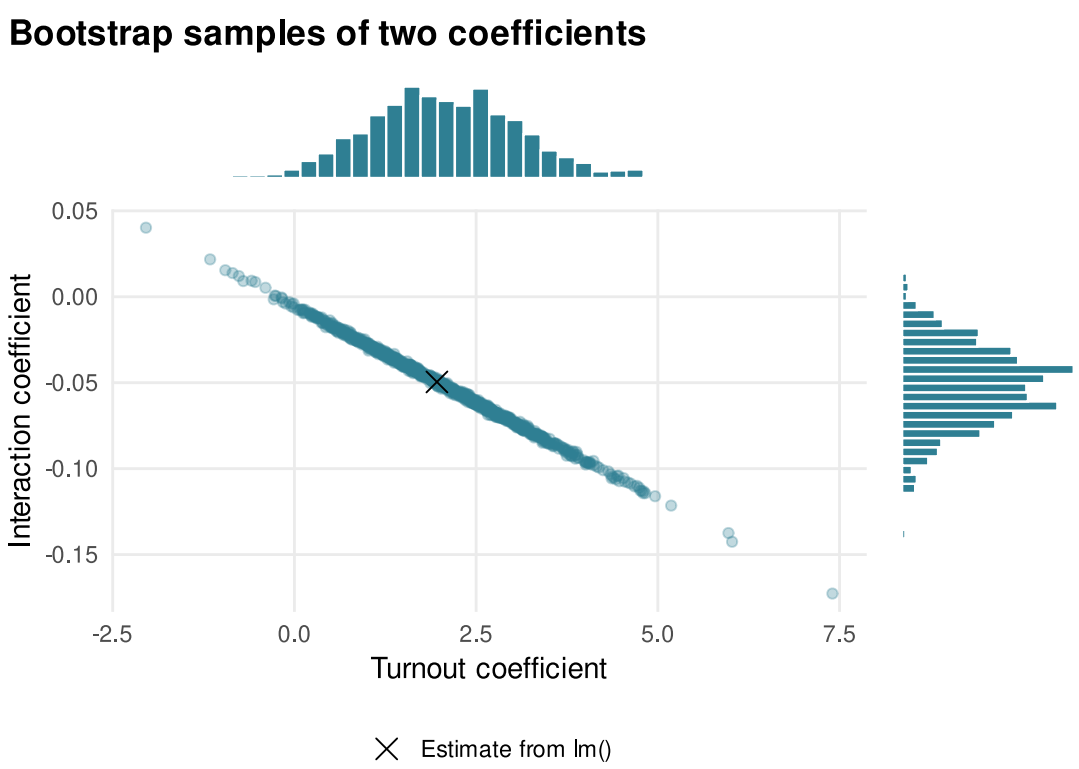
OUTPUT

```
# A tibble: 1,000 × 5
   replicate beta_0 beta_1 beta_2  beta_3
       <int>  <dbl>  <dbl>  <dbl>   <dbl>
 1         1  -338.   3.44   8.25 -0.0826
 2         2  -272.   2.41   6.69 -0.0587
 3         3  -365.   3.88   8.81 -0.0919
 4         4  -159.   1.04   4.35 -0.0302
 5         5  -199.   1.61   5.00 -0.0397
 6         6  -173.   1.43   4.54 -0.0378
 7         7  -225.   2.12   5.77 -0.0537
 8         8  -289.   2.96   7.02 -0.0705
 9         9  -305.   3.12   7.46 -0.0750
10        10  -186.   1.62   4.89 -0.0426
# ℹ 990 more rows
```

The bootstrap samples also show the covariance from Week 7. Where $\tilde{\beta}_1$ comes out larger, $\tilde{\beta}_3$ comes out more negative:

```r
lm_estimate <- tibble(
  beta_1 = age_beta_1,
  beta_3 = age_beta_3
)

boot_betas_plot <- boot_betas |>
  ggplot(aes(x = beta_1, y = beta_3)) +
  geom_point(alpha = 0.3, color = course_primary) +
  geom_point(data = lm_estimate, aes(shape = "Estimate from lm()"), size = 4) +
  scale_shape_manual(values = c("Estimate from lm()" = 4)) +
  labs(
    title = "Bootstrap samples of two coefficients",
    x     = "Turnout coefficient",
    y     = "Interaction coefficient",
    shape = NULL
  ) +
  theme_intror()
```



The histograms on the sides show each coefficient on its own: their standard deviations are the bootstrap standard errors above. Only the cloud in the middle shows how the two move together, the covariance.

Step 3. Our scenario: two counties at the mean turnout, one young (10th percentile of median age), one old (90th percentile).

```
turnout_mean <- counties |>
  summarize(mean(turnout)) |>
  pull()
age_young <- age_levels |>
  filter(percentile == 10) |>
  pull(median_age)
age_old <- age_levels |>
  filter(percentile == 90) |>
  pull(median_age)
```

Expected values and first differences

Quantity of interest (estimand): the expected AfD share at mean turnout in the young and in the old county, and the first difference between them.

Step 4. Step 3 fixed the inputs. Step 4 turns each bootstrap sample's coefficients into the quantity of interest (estimand). For each bootstrap sample r, the expected AfD share in the young county:

$$E(\text{AfD} \mid \overline{\text{Turnout}}, \text{Age}_{\text{young}})^{(r)} = \tilde{\beta}_0^{(r)} + \tilde{\beta}_1^{(r)}\overline{\text{Turnout}} + \tilde{\beta}_2^{(r)}\text{Age}_{\text{young}} + \tilde{\beta}_3^{(r)}\overline{\text{Turnout}} \cdot \text{Age}_{\text{young}}$$

the same in the old county:

$$E(\text{AfD} \mid \overline{\text{Turnout}}, \text{Age}_{\text{old}})^{(r)} = \tilde{\beta}_0^{(r)} + \tilde{\beta}_1^{(r)}\overline{\text{Turnout}} + \tilde{\beta}_2^{(r)}\text{Age}_{\text{old}} + \tilde{\beta}_3^{(r)}\overline{\text{Turnout}} \cdot \text{Age}_{\text{old}}$$

and the first difference between them:

$$\mathrm{FD}^{(r)} = E\big(\mathrm{AfD} \mid \overline{\mathrm{Turnout}}, \mathrm{Age}_{\mathrm{old}}\big)^{(r)} - E\big(\mathrm{AfD} \mid \overline{\mathrm{Turnout}}, \mathrm{Age}_{\mathrm{young}}\big)^{(r)}$$

```
boot_qoi <- boot_betas |>
  mutate(
   ev_young = beta_0 + beta_1 * turnout_mean + beta_2 * age_young + beta_3 * turnout_mean
* age_young,
    ev_old   = beta_0 + beta_1 * turnout_mean + beta_2 * age_old + beta_3 * turnout_mean
* age_old,
    fd       = ev_old - ev_young
  )
boot_qoi
```

OUTPUT

```
# A tibble: 1,000 × 8
   replicate beta_0 beta_1 beta_2  beta_3 ev_young ev_old    fd
       <int>  <dbl>  <dbl>  <dbl>   <dbl>    <dbl>  <dbl> <dbl>
 1         1  -338.   3.44   8.25 -0.0826     7.02   17.0  9.97
 2         2  -272.   2.41   6.69 -0.0587     5.96   17.3 11.3
 3         3  -365.   3.88   8.81 -0.0919     6.87   16.1  9.18
 4         4  -159.   1.04   4.35 -0.0302     6.69   17.1 10.5
 5         5  -199.   1.61   5.00 -0.0397     7.17   17.2 10.1
 6         6  -173.   1.43   4.54 -0.0378     7.29   15.8  8.51
 7         7  -225.   2.12   5.77 -0.0537     7.41   16.0  8.57
 8         8  -289.   2.96   7.02 -0.0705     6.88   15.3  8.41
 9         9  -305.   3.12   7.46 -0.0750     7.25   16.2  8.94
10        10  -186.   1.62   4.89 -0.0426     7.36   15.8  8.39
# ℹ 990 more rows
```

Step 5. The mean and the 2.5% and 97.5% quantiles of each distribution give the estimate and its 95% confidence interval. One long tibble holds all three quantities:

```r
boot_qoi_long <- boot_qoi |>
  pivot_longer(c(ev_young, ev_old, fd), names_to = "quantity", values_to = "value") |>
  mutate(quantity = fct_inorder(quantity))
boot_qoi_long
```

OUTPUT

```
# A tibble: 3,000 × 7
   replicate beta_0 beta_1 beta_2  beta_3 quantity value
       <int>  <dbl>  <dbl>  <dbl>   <dbl> <fct>    <dbl>
 1         1  -338.   3.44   8.25 -0.0826 ev_young  7.02
 2         1  -338.   3.44   8.25 -0.0826 ev_old   17.0
 3         1  -338.   3.44   8.25 -0.0826 fd         9.97
 4         2  -272.   2.41   6.69 -0.0587 ev_young  5.96
 5         2  -272.   2.41   6.69 -0.0587 ev_old   17.3
 6         2  -272.   2.41   6.69 -0.0587 fd       11.3
 7         3  -365.   3.88   8.81 -0.0919 ev_young  6.87
 8         3  -365.   3.88   8.81 -0.0919 ev_old   16.1
 9         3  -365.   3.88   8.81 -0.0919 fd         9.18
10         4  -159.   1.04   4.35 -0.0302 ev_young  6.69
# ℹ 2,990 more rows
```

```r
qoi_summary <- boot_qoi_long |>
  group_by(quantity) |>
  summarize(
    estimate = mean(value),
    lower    = quantile(value, 0.025),
    upper    = quantile(value, 0.975)
  )
qoi_summary
```

OUTPUT

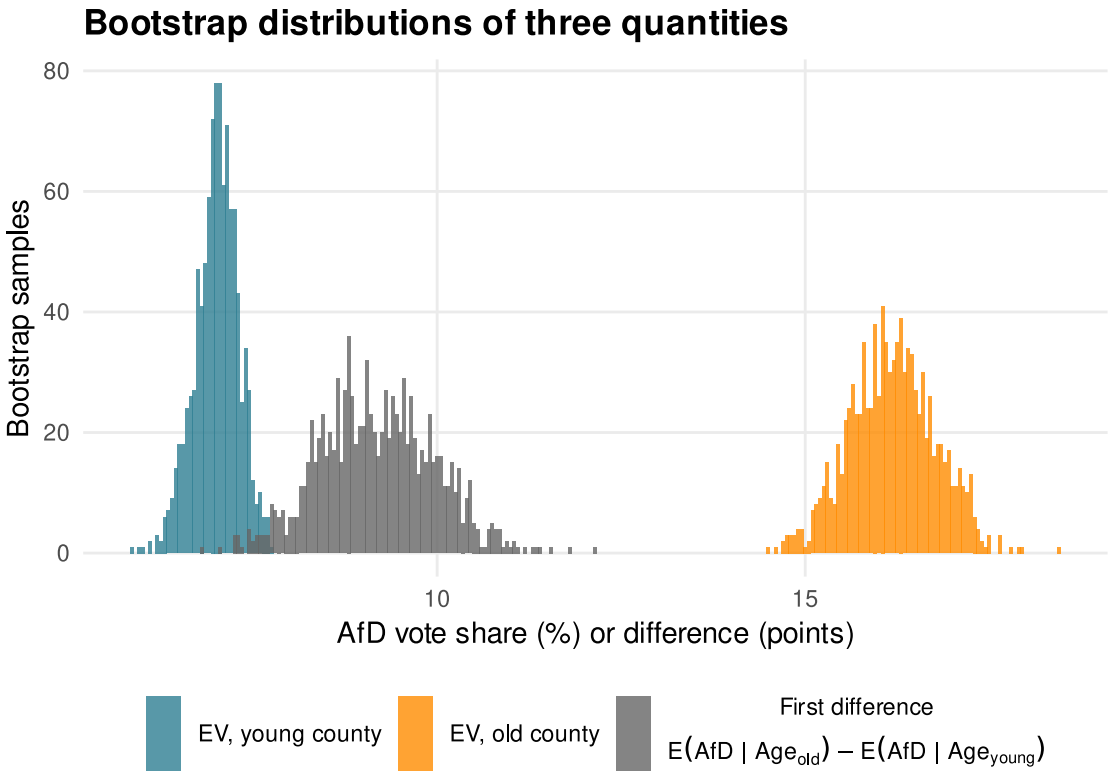
```
# A tibble: 3 × 4
  quantity estimate lower upper
  <fct>       <dbl> <dbl> <dbl>
1 ev_young     7.01  6.40  7.56
2 ev_old      16.2  15.1  17.3
3 fd           9.18  7.72 10.8
```

```r
fd_summary <- qoi_summary |>
  filter(quantity == "fd")
fd_estimate <- fd_summary |>
  pull(estimate)
fd_lower <- fd_summary |>
  pull(lower)
fd_upper <- fd_summary |>
  pull(upper)
```

The three distributions:

```r
qoi_colors <- c(
  ev_young = course_primary,
  ev_old   = "darkorange",
  fd       = "grey40"
)
qoi_labels <- c(
  ev_young = "EV, young county",
  ev_old   = "EV, old county",
  fd       = expression(atop("First difference", E(AfD ~ "|" ~ Age[old]) - E(AfD ~ "|"
~ Age[young])))
)

qoi_plot <- boot_qoi_long |>
  ggplot(aes(x = value, fill = quantity)) +
  geom_histogram(binwidth = 0.05, alpha = 0.8, position = "identity") +
  scale_fill_manual(values = qoi_colors, labels = qoi_labels) +
  labs(
    title = "Bootstrap distributions of three quantities",
    x     = "AfD vote share (%) or difference (points)",
    y     = "Bootstrap samples",
    fill  = NULL
  ) +
  theme_intror()
qoi_plot
```



At average turnout, an old county is expected to vote 9.2 points more AfD than a young one, with a 95% interval from 7.7 to 10.8. This describes counties as they are, not the effect of a county growing older.

Comparison with `marginaleffects`

`predictions()` computes expected values, `comparisons()` first differences. By default they use the delta method, an analytical approximation built on $\hat{\boldsymbol{V}}$. `inferences()` replaces that with the bootstrap (`"boot"`) or with the lecture's draws from the multivariate Normal (`"simulation"`):

```
evs_delta <- predictions(
  age_model,
  newdata = datagrid(turnout = turnout_mean, median_age = c(age_young, age_old)),
  df      = "residual"
)
evs_boot <- evs_delta |>
  inferences(method = "boot", R = n_boot)
evs_simulation <- evs_delta |>
  inferences(method = "simulation", R = n_boot)
evs_boot
```

OUTPUT

```
 turnout median_age Estimate 2.5 % 97.5 %
    76.3       42.4     7.02  6.44   7.53
    76.3       47.6    16.17 15.07  17.33

Type: response
```

```
evs_simulation
```

OUTPUT

```
 turnout median_age Estimate 2.5 % 97.5 %
    76.3       42.4     7.02  6.34   7.66
    76.3       47.6    16.17 15.42  16.92

Type: response
```

```
fds_delta <- comparisons(
  age_model,
  variables = list(median_age = c(age_young, age_old)),
  newdata   = datagrid(turnout = turnout_mean),
  df        = "residual"
)
fds_boot <- fds_delta |>
  inferences(method = "boot", R = n_boot)
fds_simulation <- fds_delta |>
  inferences(method = "simulation", R = n_boot)
fds_boot
```

OUTPUT

```
 turnout Estimate 2.5 % 97.5 %
    76.3     9.15  7.66   10.6

Term: median_age
Type: response
Comparison: 47.562 - 42.448
```

```
fds_simulation
```

OUTPUT

```
 turnout Estimate 2.5 % 97.5 %
    76.3     9.15  8.02   10.2

Term: median_age
Type: response
Comparison: 47.562 - 42.448
```

All in one table, with our own bootstrap first:

OUTPUT

```
# A tibble: 12 × 5
   method                      quantity         estimate lower upper
   <chr>                       <chr>               <dbl> <dbl> <dbl>
 1 Bootstrap by hand           EV, old county      16.2  15.1  17.3
 2 marginaleffects: boot       EV, old county      16.2  15.1  17.3
 3 marginaleffects: simulation EV, old county      16.2  15.4  16.9
 4 marginaleffects: delta      EV, old county      16.2  15.4  16.9
 5 Bootstrap by hand           EV, young county     7.01  6.40  7.56
 6 marginaleffects: boot       EV, young county     7.02  6.44  7.53
 7 marginaleffects: simulation EV, young county     7.02  6.34  7.66
 8 marginaleffects: delta      EV, young county     7.02  6.36  7.68
 9 Bootstrap by hand           First difference     9.18  7.72 10.8
10 marginaleffects: boot       First difference     9.15  7.66 10.6
11 marginaleffects: simulation First difference     9.15  8.02 10.2
12 marginaleffects: delta      First difference     9.15  8.00 10.3
```

> **Note**
>
> All four lie within a few tenths of a point of each other. Bootstrap and simulation don't have to agree: the simulation takes $\hat{\boldsymbol{V}}$ from the OLS formula, which assumes the same error variance for every county. The bootstrap only uses the counties themselves. Where some counties scatter much more than others, its intervals come out wider.

> **A Bayesian parallel**
>
> The five steps resemble how Bayesian analysts work. They draw the coefficients from a *posterior distribution*, which describes how plausible each value of β is given the data. Then they compute every quantity once per draw and summarize the draws, as in Steps 4 and 5. Expected values computed this way are what Bayesian analysts call expected values of the posterior predictive distribution. Adding a draw of the error term, as we do for predicted values below, gives **posterior predictive draws**. Taking the mean of the draws as the point estimate is the posterior mean.
>
> The lecture's draws from $\mathcal{MVN}\left(\hat{\beta}, \hat{\boldsymbol{V}}\right)$ are therefore often called *pseudo-Bayesian*: they are read like a posterior without a Bayesian model behind them. Our bootstrap is the frequentist counterpart. Its draws show how the estimate would vary from sample to sample, not how plausible each β is.

Expected values over a range

Quantity of interest (estimand): the expected AfD share at every turnout value, in the young and in the old county.

As with Week 7's lines at different ages, a scenario can run over a whole range: turnout from its minimum to its maximum, in a young and an old county. Step 4 and Step 5 for each of the 50 turnout values:

```
range_scenarios <- expand_grid(
  turnout    = turnout_grid,
  median_age = c(age_young, age_old)
)
```

$$E(\text{AfD} \mid \text{Turnout}, \text{Age})^{(r)} = \tilde{\beta}_0^{(r)} + \tilde{\beta}_1^{(r)}\ \text{Turnout} + \tilde{\beta}_2^{(r)}\ \text{Age} + \tilde{\beta}_3^{(r)}\ \text{Turnout} \cdot \text{Age}$$

```
range_evs <- expand_grid(boot_betas, range_scenarios) |>
  mutate(ev = beta_0 + beta_1 * turnout + beta_2 * median_age + beta_3 * turnout *
median_age) |>
  group_by(turnout, median_age) |>
  summarize(
    estimate = mean(ev),
    lower    = quantile(ev, 0.025),
    upper    = quantile(ev, 0.975),
    .groups  = "drop"
  ) |>
  mutate(scenario = if_else(median_age == age_young, "ev_young", "ev_old"))
range_evs
```

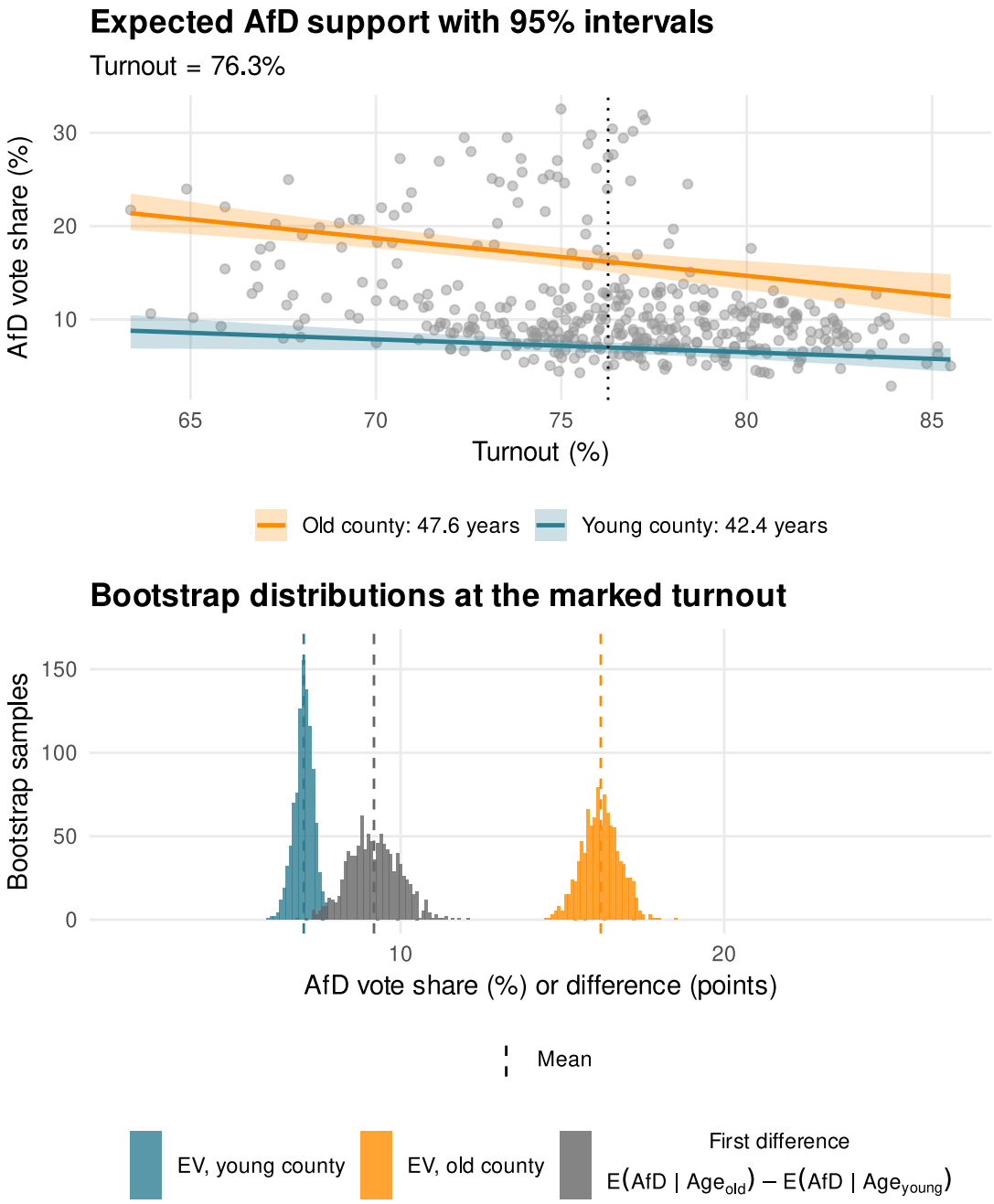
OUTPUT

```
# A tibble: 100 × 6
   turnout median_age estimate lower upper scenario
     <dbl>      <dbl>    <dbl> <dbl> <dbl> <chr>
 1    63.4       42.4     8.80  6.90 10.5  ev_young
 2    63.4       47.6    21.4  19.6  23.5  ev_old
 3    63.8       42.4     8.74  6.88 10.4  ev_young
 4    63.8       47.6    21.2  19.5  23.3  ev_old
 5    64.3       42.4     8.67  6.87 10.2  ev_young
 6    64.3       47.6    21.0  19.3  23.0  ev_old
 7    64.7       42.4     8.61  6.87 10.1  ev_young
 8    64.7       47.6    20.8  19.2  22.8  ev_old
 9    65.2       42.4     8.55  6.85  9.97 ev_young
10    65.2       47.6    20.7  19.1  22.5  ev_old
# ℹ 90 more rows
```

```
range_labels <- c(
  ev_young = str_glue("Young county: {round(age_young, 1)} years"),
  ev_old   = str_glue("Old county: {round(age_old, 1)} years")
)

range_evs_plot <- range_evs |>
  ggplot(aes(x = turnout, color = scenario, fill = scenario)) +
  geom_point(data = counties, aes(y = afd), color = "grey60", fill = NA, alpha = 0.5) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.25, color = NA) +
  geom_line(aes(y = estimate), linewidth = 1) +
 scale_color_manual(values = qoi_colors, labels = range_labels, aesthetics = c("color",
"fill")) +
  labs(
    title = "Expected AfD support with 95% intervals",
    x     = "Turnout (%)",
    y     = "AfD vote share (%)",
    color = NULL,
    fill  = NULL
  ) +
  theme_intror()
```

The bootstrap distributions from above are a cut through this plot at mean turnout, the vertical line. The animation moves the line along the turnout axis, with the distributions at that turnout below:



As the line moves to higher turnout, the old county's expected value falls faster than the young one's, so the first difference shrinks: that is the interaction. The distributions are narrow near average turnout and wide at the edges of the data, where few counties back the estimate.

The same with `marginaleffects`

`predictions()` with the turnout grid and both ages, bootstrapped, gives the bands of `range_evs`. Its estimates come from the original fit, not the mean of the bootstrap samples:

```
predictions(
  age_model,
  newdata = datagrid(turnout = turnout_grid, median_age = c(age_young, age_old))
) |>
  inferences(method = "boot", R = n_boot)
```

Predicted values

Quantity of interest: the AfD share of a single new county with the scenario's values. It is not an estimand in the strict sense: it stays random even with the true β.

A predicted value adds the fundamental uncertainty: each expected value gets a random error from $\mathcal{N}(0, \hat{\sigma}^2)$, with $\hat{\sigma}$ the residual standard deviation, as in Week 7's single-predictor example:

```
sigma_hat <- age_model |>
  glance() |>
  pull(sigma)
sigma_hat
```

OUTPUT

```
[1] 4.216819
```

$$\text{PV}^{(r)} = E(\text{AfD} \mid \text{Turnout}, \text{Age})^{(r)} + \tilde{\varepsilon}^{(r)}, \quad \tilde{\varepsilon}^{(r)} \sim \mathcal{N}(0, \hat{\sigma}^2)$$

```
boot_pvs <- boot_qoi |>
  mutate(
    pv_young = ev_young + rnorm(n(), 0, sigma_hat),
    pv_old   = ev_old + rnorm(n(), 0, sigma_hat)
  )
```

```
pv_summary <- boot_pvs |>
  pivot_longer(c(pv_young, pv_old), names_to = "quantity", values_to = "value") |>
  group_by(quantity) |>
  summarize(
    estimate = mean(value),
    lower    = quantile(value, 0.025),
    upper    = quantile(value, 0.975)
  )
pv_summary
```

OUTPUT

```
# A tibble: 2 × 4
  quantity estimate lower upper
  <chr>       <dbl> <dbl> <dbl>
1 pv_old      16.1   7.62  24.9
2 pv_young     6.84 -1.80  15.1
```

`augment()` gives the analytical prediction interval, under the same Normal assumption as our error draws:

```
scenarios <- tibble(
  scenario   = c("young", "old"),
  turnout    = turnout_mean,
  median_age = c(age_young, age_old)
)

age_model |>
  augment(newdata = scenarios, interval = "prediction") |>
  select(scenario, .fitted, .lower, .upper)
```

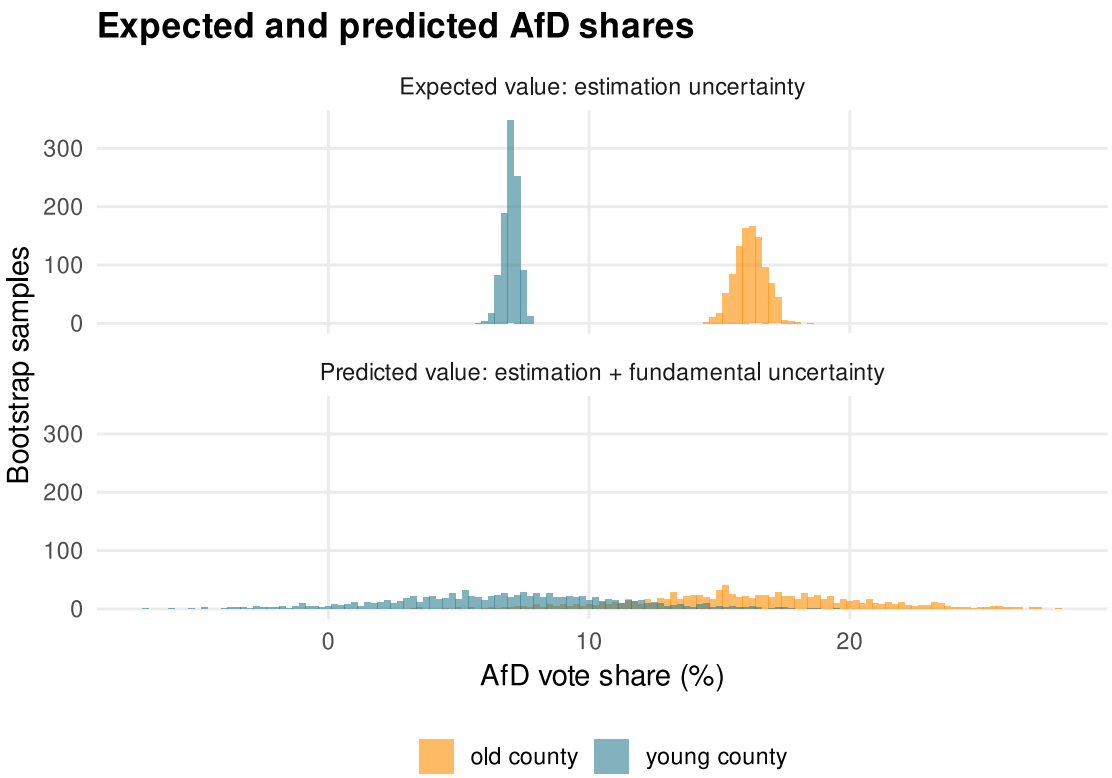
OUTPUT

```
# A tibble: 2 × 4
  scenario .fitted .lower .upper
  <chr>      <dbl>  <dbl>  <dbl>
1 young       7.02  -1.30   15.3
2 old        16.2    7.84   24.5
```

Both give similar intervals. The same scenario, both kinds of values side by side:

```
ev_pv_draws <- boot_pvs |>
  pivot_longer(c(ev_young, ev_old, pv_young, pv_old), names_to = c("type", "scenario"),
names_sep = "_", values_to = "value") |>
  mutate(
    type = if_else(
      type == "ev",
      "Expected value: estimation uncertainty",
      "Predicted value: estimation + fundamental uncertainty"
    ),
    scenario = str_glue("{scenario} county")
  )
```

```r
ev_pv_draws |>
  ggplot(aes(x = value, fill = scenario)) +
  geom_histogram(binwidth = 0.25, alpha = 0.6, position = "identity") +
  facet_wrap(~type, ncol = 1) +
    scale_fill_manual(values = c("young county" = course_primary, "old county" =
"darkorange")) +
  labs(
    title = "Expected and predicted AfD shares",
    x     = "AfD vote share (%)",
    y     = "Bootstrap samples",
    fill  = NULL
  ) +
  theme_intror()
```



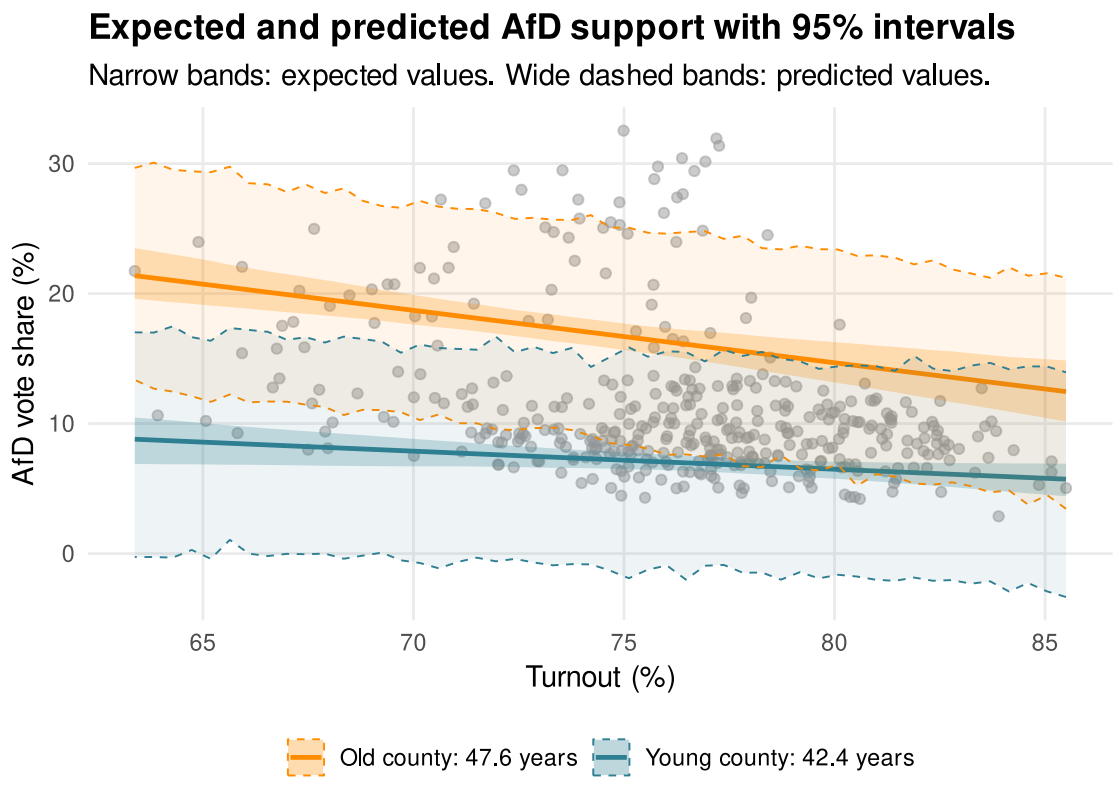
We are fairly sure where the *average* old county lies. A single old county can lie far from it: the predicted values of young and old counties overlap widely.

Only the upper interval is a confidence interval: it is about a fixed quantity, the expected value, and shrinks towards zero width with more data. The lower one is a **prediction interval** for a single new county. With more data it shrinks only to about $\pm 1.96\hat{\sigma}$, the fundamental uncertainty. For the young county it even reaches below zero, as in Week 7′s single-predictor example.

The same over the whole turnout range: each expected value from `range_evs` gets an error draw, then we take the quantiles.

```r
range_pvs <- expand_grid(boot_betas, range_scenarios) |>
  mutate(
    ev = beta_0 + beta_1 * turnout + beta_2 * median_age + beta_3 * turnout * median_age,
     pv = ev + rnorm(n(), 0, sigma_hat)
  ) |>
  group_by(turnout, median_age) |>
  summarize(
    lower   = quantile(pv, 0.025),
    upper   = quantile(pv, 0.975),
    .groups = "drop"
  ) |>
  mutate(scenario = if_else(median_age == age_young, "ev_young", "ev_old"))
```

```r
range_evs_plot +
  geom_ribbon(data = range_pvs, aes(ymin = lower, ymax = upper), alpha = 0.08, linetype
= "dashed", linewidth = 0.4) +
  labs(
    title    = "Expected and predicted AfD support with 95% intervals",
    subtitle = "Narrow bands: expected values. Wide dashed bands: predicted values."
  )
```



The prediction bands are several times wider than the expected-value bands, and almost as wide in the middle of the turnout range as at its edges: the fundamental uncertainty is the same everywhere, only the estimation uncertainty grows towards the edges.

From the bootstrap to Monte Carlo

The bootstrap made many samples out of the one sample we have, to approximate the sampling distribution of $\hat{\beta}$. It cannot tell us whether that distribution is centered on the true β: with real data, nobody knows the truth. To see what OLS does when its assumptions hold, and when they don't, we need data whose truth we know. So we create it ourselves.

The classical linear model assumptions

The lecture's assumptions, with what goes wrong without them:

Assumption	Notation	Intuitively	If it is violated
1. Linearity in parameters	$Y = \beta_0 + \beta_1 x_1 + ... + \beta_k x_k + \varepsilon$	Effects add up; curves are possible only through transformed variables	$\hat{\beta}$ describes the wrong relationship; biased
2. Zero conditional mean	$E(\varepsilon \mid x_1, ..., x_k) = 0$	Nothing in the error term is related to the predictors	$\hat{\beta}$ biased, e.g. by an omitted confounder
3. Homoskedasticity	$\mathrm{Var}(\varepsilon \mid x_1, ..., x_k) = \sigma_\varepsilon^2$	Observations scatter equally far around the line everywhere	$\hat{\beta}$ unbiased, but the OLS standard errors are wrong
4. Independence	$\mathrm{Cov}(\varepsilon_i, \varepsilon_j \mid x_i, x_j) = 0, \quad i \neq j$	Each observation brings new information, no clusters	$\hat{\beta}$ unbiased, standard errors usually too small
5. Non-stochastic X	x_i fixed in repeated samples, measured without error	We measure the predictors exactly	Coefficient of the noisy variable biased towards zero, others biased too
6. Normality	$\varepsilon_i \sim \mathcal{N}(0, \sigma_\varepsilon^2)$	Matters for exact inference in small samples only	$\hat{\beta}$ unbiased; t-tests and intervals only approximate in small samples
No perfect collinearity	No predictor is an exact linear combination of the others	Each predictor has to vary on its own	Perfect: $\hat{\beta}$ cannot be computed. Near-perfect: unbiased, but large standard errors

Where each one gets simulated below:

- Linearity (1): *Curved education effect (Try it)*

- Zero conditional mean (2): Omitted variable
- Homoskedasticity (3): Heteroskedasticity
- Independence (4): *Clustered surveys (Try it)*
- Non-stochastic X (5): Measurement error
- Normality (6): *Skewed errors (Try it)*
- No perfect collinearity: Multicollinearity

Assumptions 1 and 2 make OLS **unbiased**: $E\left(\hat{\beta}_j\right) = \beta_j$. With 3 and 4 as well, OLS has the smallest variance among the linear unbiased estimators (the Gauss–Markov theorem). The last row is not in the lecture's list but OLS needs it to compute anything; how close a data set comes to violating it decides how precise the estimates are.

Monte Carlo simulation

A **Monte Carlo simulation** studies an estimator by applying it to many random samples from a population we created ourselves. Because we set the true coefficients, we can compare every estimate with the truth. Why bother:

- **Real data never reveal the truth.** In a simulation we know β, so we can see whether $\hat{\beta}$ hits it on average (bias) and how far it scatters (the standard error).
- **Assumptions become visible.** Breaking one assumption at a time shows what it does to the point estimate and to the standard error.
- **Checking our tools.** We can see whether the standard error `lm()` reports matches the actual spread of the estimates, and whether a fix such as the bootstrap repairs it.
- **Planning and robustness.** How many respondents does a survey need? How strong would an unmeasured confounder have to be to explain an effect away? A simulation answers both before any data are collected.

The workflow has five steps:

1. **Generate the population**: choose the true coefficients, simulate the predictors, and compute the outcome from the true model plus a random error.
2. **Draw a sample** of n observations from the population.
3. **Estimate** the model on the sample and record $\hat{\beta}$ and its standard error.
4. **Repeat** steps 2 and 3 R times. The R estimates form the **sampling distribution** of $\hat{\beta}$.
5. **Compare** with the truth: the mean of the estimates against β (bias), and their standard deviation against the reported standard errors.

> **i** Note
>
> The lecture keeps X fixed and only redraws the error term in step 2. We draw whole respondents from a population, as a survey does, so X changes from sample to sample too. For the questions below, both give the same answers.

The population

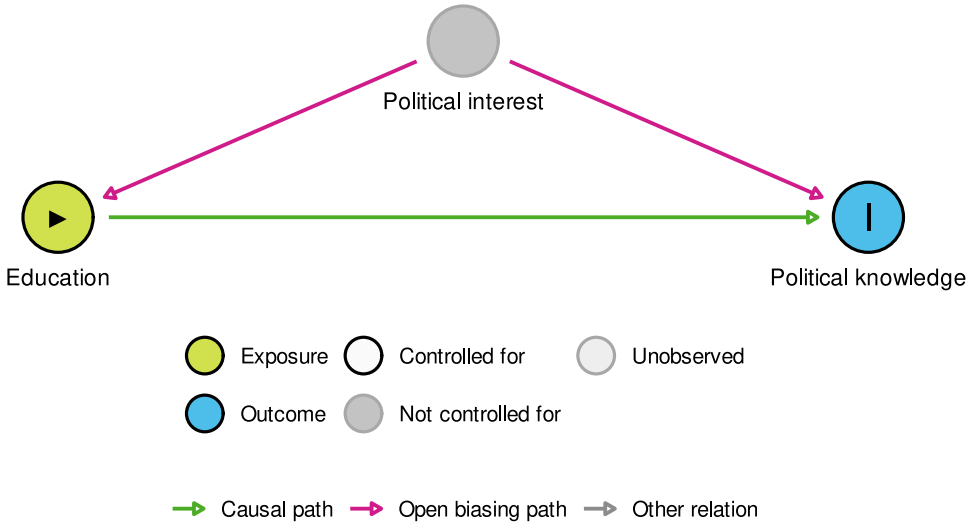
Quantity of interest (estimand): the effect of one more year of education on a voter's political knowledge, β_1 below.

Political scientists ask whether education makes citizens better informed about politics. Education is not randomly assigned, though: people who are interested in politics early on also tend to stay longer in education. The DAG:

```
source("include/plot_dag.R")

knowledge_labels <- tribble(
  ~name,       ~label,
  "education", "Education",
  "interest",  "Political interest",
  "knowledge", "Political knowledge"
)

dagitty('dag {
  education [exposure, pos="0,0"]
  knowledge [outcome, pos="2,0"]
  interest [pos="1,-1"]
  education -> knowledge
  interest -> education
  interest -> knowledge
}') |>
  plot_dag(labels = knowledge_labels)
```



- **Education → Knowledge**: school and university teach how politics works and train people to follow the news.
- **Interest → Education**: interest in politics is formed early, in the family, and interested teenagers more often go on to a longer education.
- **Interest → Knowledge**: interested people follow politics and so know more about it, whatever their education.

Political interest is a **confounder**: it opens the backdoor path Education ← Interest → Knowledge. To estimate β_1, the model must include it:

$$\text{Knowledge}_i = \beta_0 + \beta_1 \text{Educ}_i + \beta_2 \text{Interest}_i + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_\varepsilon^2)$$

Step 1: generate the population. These are the true values. In real research they are unknown; here we set them. Political interest is measured on a scale from 0 to 10. Education is measured in years and depends on interest, with ν_i for everything else that drives schooling; nobody has fewer than 8 years. `knowledge` is a score from a battery of knowledge questions:

```
n_population   <- 100000
# interest
interest_mean  <- 5
interest_sd    <- 1.8
interest_min   <- 0
interest_max   <- 10
# education
delta_0        <- 9
delta_interest <- 0.6
sigma_nu       <- 2
education_min  <- 8
# knowledge
beta_0         <- 5
beta_education <- 2.5
beta_interest  <- 3
sigma_epsilon  <- 8
```

The population line by line, in the order the code builds it. At the end, voters whose interest falls outside 0–10 or who have fewer than 8 years of education are dropped (truncation):

$$\begin{aligned} \text{Interest}_i &\sim \mathcal{N}(5, 1.8^2) \\ \text{Educ}_i &= \delta_0 + \delta_1 \text{Interest}_i + \nu_i, & \nu_i &\sim \mathcal{N}(0, \sigma_\nu^2) \\ \text{Knowledge}_i &= \beta_0 + \beta_1 \text{Educ}_i + \beta_2 \text{Interest}_i + \varepsilon_i, & \varepsilon_i &\sim \mathcal{N}(0, \sigma_\varepsilon^2) \end{aligned}$$

The same written as distributions: each variable is Normal, with the systematic part of its equation as the mean. The lecture calls these the *stochastic* and the *systematic component*:

$$\begin{aligned} \text{Interest}_i &\sim \mathcal{N}(5, 1.8^2) \\ \text{Educ}_i &\sim \mathcal{N}(\delta_0 + \delta_1 \text{Interest}_i, \sigma_\nu^2) \\ \text{Knowledge}_i &\sim \mathcal{N}(\beta_0 + \beta_1 \text{Educ}_i + \beta_2 \text{Interest}_i, \sigma_\varepsilon^2) \end{aligned}$$

A Bayesian parallel

These three lines are a complete **generative model**: they say how the data come about, step by step. A Bayesian analysis writes down exactly the same kind of model, and adds a *prior* distribution for every parameter (β, δ, σ).

The two then run in opposite directions. A Monte Carlo simulation fixes the parameters and generates data, to see what an estimator does with them. Bayesian inference starts from the observed data and asks which parameter values could have generated them: the *posterior*. Before fitting, Bayesian analysts often run our simulation too, with parameters drawn from the priors instead of fixed, to check that the model produces plausible data. This is called a *prior predictive check*.

The population, one row per voter:

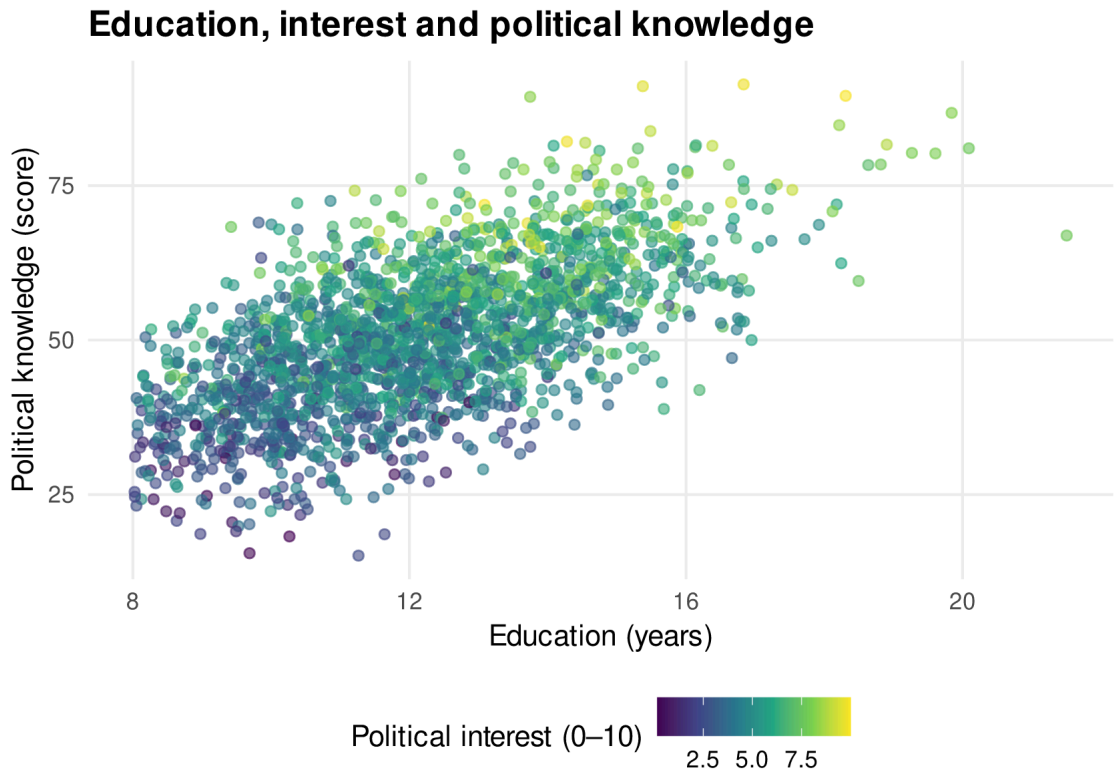
```
population <- tibble(interest = rnorm(n_population, interest_mean, interest_sd)) |>
  mutate(
    education = delta_0 + delta_interest * interest + rnorm(n(), 0, sigma_nu),
     knowledge = beta_0 + beta_education * education + beta_interest * interest +
rnorm(n(), 0, sigma_epsilon)
  ) |>
  filter(
    between(interest, interest_min, interest_max),
    education >= education_min
  ) |>
  mutate(voter_id = row_number(), .before = 1)
population
```

OUTPUT

```
# A tibble: 95,576 × 4
   voter_id interest education knowledge
      <int>    <dbl>     <dbl>     <dbl>
 1        1     4.09      14.6      66.9
 2        2     8.32      10.8      79.9
 3        3     7.07      10.8      38.8
 4        4     3.51      12.7      38.9
 5        5     7.01      13.4      63.7
 6        6     8.10      14.8      80.8
 7        7     7.79      16.5      69.2
 8        8     1.89       8.36     22.6
 9        9     3.81      10.2      33.1
10       10     2.06       9.83     37.2
# ℹ 95,566 more rows
```

A first look at 2,000 of the voters:

```r
population |>
  slice_sample(n = 2000) |>
  ggplot(aes(x = education, y = knowledge, color = interest)) +
  geom_point(alpha = 0.6) +
  scale_color_viridis_c() +
  labs(
    title = "Education, interest and political knowledge",
    x     = "Education (years)",
    y     = "Political knowledge (score)",
    color = "Political interest (0–10)"
  ) +
  theme_intror()
```



Knowledge rises with education, and at every level of education the more interested voters (yellow) know more.

The true coefficients in one tibble, for step 5:

```r
truth <- tibble(
  term       = c("education", "interest"),
  true_value = c(beta_education, beta_interest)
)
truth
```

```
# A tibble: 2 × 2
  term      true_value
  <chr>          <dbl>
1 education        2.5
2 interest         3
```

No violation: the sampling distribution

Quantity of interest (estimand): $\beta_1 = 2.5$, the effect of a year of education, and $\beta_2 = 3$, the effect of a point of interest.

Step 2: draw samples. Each sample is a survey of 500 voters, and we draw 1,000 of them. We draw all respondents at once and number the samples with `sample_id`. With about 100,000 voters, drawing with replacement almost never picks the same voter twice in a survey. `mc_draws` keeps only which voter is in which survey, so that every round below can use the same surveys:

```r
n_sample <- 500
n_sims   <- 1000
```

```r
mc_draws <- population |>
  slice_sample(n = n_sample * n_sims, replace = TRUE) |>
  mutate(sample_id = rep(1:n_sims, each = n_sample)) |>
  select(sample_id, voter_id)

mc_samples <- mc_draws |>
  left_join(population, by = "voter_id")
mc_samples
```

OUTPUT

```
# A tibble: 500,000 × 5
   sample_id voter_id interest education knowledge
       <int>    <int>    <dbl>     <dbl>     <dbl>
 1         1    63565    4.15      11.0       53.6
 2         1    84783    6.64       9.13      52.5
 3         1     3295    5.61      13.8       65.9
 4         1    31357    7.49      13.3       68.4
 5         1    37498    6.27      14.4       70.0
 6         1    57012    4.19      13.8       47.9
 7         1    14204    2.75      13.4       56.5
 8         1    19663    5.58      13.1       44.6
 9         1    71564    0.991      9.35      17.7
10         1    11288    3.48      14.2       38.9
# ℹ 499,990 more rows
```

When a section below adds a column to the population, the same join brings it into the surveys.

Steps 3 and 4: estimate, and repeat. Every round below fits a model on each of the 1,000 samples; only the formula changes. So we write one function for a round. It works like the bootstrap's step 2 above: `rowwise()` goes through one sample per row, filters its respondents, fits the model and tidies it. `pivot_wider()` puts estimates and standard errors side by side in one row, and `pivot_longer()` at the end turns them back into one row per sample and coefficient. The variant columns of the population (`interest_reported`, …) stand in for the true variable, so `str_remove()` cuts their suffix and gives their coefficients the true variable's name:

```r
simulate_round <- function(samples, formula, round_name) {
  samples |>
    distinct(sample = sample_id) |>
    rowwise() |>
    mutate(
      samples |>
        filter(sample_id == sample) |>
        lm(formula, data = _) |>
        tidy() |>
        select(term, estimate, std.error) |>
        pivot_wider(names_from = term, values_from = c(estimate, std.error))
    ) |>
    ungroup() |>
    pivot_longer(
      -sample,
      names_to      = c(".value", "term"),
      names_pattern = "(estimate|std.error)_(.*)"
    ) |>
    mutate(
      term  = str_remove(term, "_[a-z]+$"),
      round = round_name
    ) |>
    rename(sample_id = sample)
}
```

The correctly specified model, estimated on every sample:

$$\widehat{\text{Knowledge}}_i = \hat{\beta}_0 + \hat{\beta}_1 \text{Educ}_i + \hat{\beta}_2 \text{Interest}_i$$

```r
baseline <- mc_samples |>
  simulate_round(knowledge ~ education + interest, "No violation")
baseline
```

OUTPUT

```
# A tibble: 3,000 × 5
   sample_id term        estimate std.error round
       <int> <chr>          <dbl>     <dbl> <chr>
 1         1 (Intercept)    5.65      2.06  No violation
 2         1 education      2.31      0.178 No violation
 3         1 interest       3.31      0.233 No violation
 4         2 (Intercept)    5.58      2.15  No violation
 5         2 education      2.33      0.196 No violation
 6         2 interest       3.33      0.239 No violation
 7         3 (Intercept)    0.816     2.13  No violation
 8         3 education      2.81      0.191 No violation
 9         3 interest       3.11      0.228 No violation
10         4 (Intercept)    4.80      2.24  No violation
# ℹ 2,990 more rows
```

Step 5: compare with the truth. For each coefficient, three numbers answer two questions. Is the estimator unbiased?

$$\text{Bias}\left(\hat{\beta}_j\right) = \overline{\hat{\beta}_j} - \beta_j, \quad \overline{\hat{\beta}_j} = \frac{1}{R}\sum_{r=1}^{R} \hat{\beta}_j^{(r)}$$

And is the standard error that `lm()` reports right? The actual spread of the estimates is their standard deviation across samples; the reported one is the mean of the `std.error` column:

$$\text{sd}\left(\hat{\beta}_j^{(1)}, \ldots, \hat{\beta}_j^{(R)}\right) \quad \text{vs.} \quad \overline{\text{SE}\left(\hat{\beta}_j\right)}$$

The same comparison is needed in every round, so it gets a function too:

```
summarize_round <- function(results, truth) {
  results |>
    inner_join(truth, by = "term") |>
    group_by(round, term, true_value) |>
    summarize(
      mean_estimate = mean(estimate),
      sd_estimate   = sd(estimate),
      mean_se       = mean(std.error),
      .groups       = "drop"
    ) |>
    mutate(bias = mean_estimate - true_value, .after = mean_estimate)
}
```

```
baseline_summary <- baseline |>
  summarize_round(truth)
baseline_summary
```

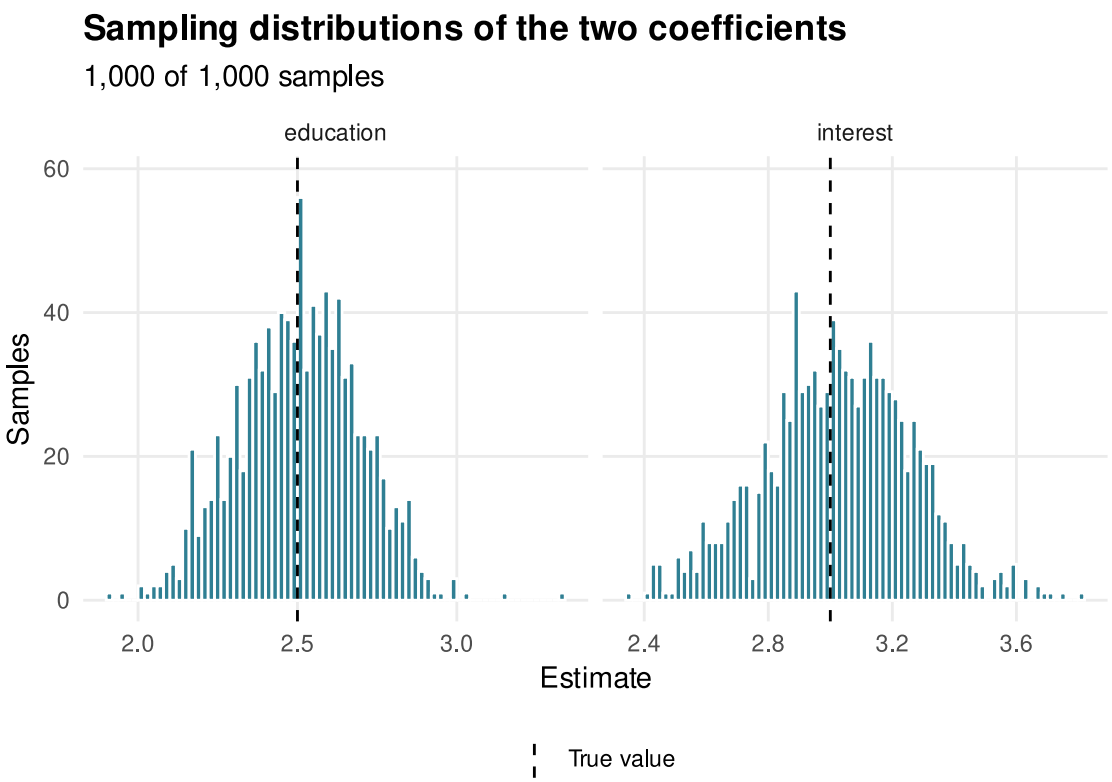
OUTPUT

```
# A tibble: 2 × 7
  round        term      true_value mean_estimate    bias sd_estimate mean_se
  <chr>        <chr>          <dbl>         <dbl>   <dbl>       <dbl>   <dbl>
1 No violation education        2.5          2.51 0.00551       0.189   0.193
2 No violation interest         3            3.03 0.0258        0.238   0.230
```

```
baseline_education <- baseline_summary |>
  filter(term == "education")
baseline_mean <- baseline_education |>
  pull(mean_estimate)
baseline_sd <- baseline_education |>
  pull(sd_estimate)
baseline_se <- baseline_education |>
  pull(mean_se)
```

Across 1,000 surveys, the education coefficient averages 2.506, against a true value of 2.5: OLS is unbiased. Its estimates scatter with a standard deviation of 0.189, and the standard error `lm()` reports is 0.193 on average: when the assumptions hold, the formula gets the spread right.

The sampling distribution builds up one survey at a time. The animation adds more and more samples:



Both distributions are centered on the true values (dashed). Each survey misses the truth a little, but not in any particular direction. This is the benchmark for every round below.

The plot for each round compares the sampling distribution of the education coefficient with this one, so it gets a function as well:

```
plot_rounds <- function(results, truth, title) {
  results |>
    filter(term == "education") |>
    inner_join(truth, by = "term") |>
    mutate(round = fct_inorder(round)) |>
    ggplot(aes(x = estimate, fill = round)) +
    geom_histogram(bins = 50, alpha = 0.6, position = "identity") +
    geom_vline(aes(xintercept = true_value, linetype = "True value")) +
    scale_fill_manual(values = c(course_primary, "grey60")) +
    scale_linetype_manual(values = c("True value" = "dashed")) +
    labs(
      title    = title,
      x        = "Estimate",
      y        = "Samples",
      fill     = NULL,
      linetype = NULL
    ) +
    theme_intror()
}
```

Omitted variable

Quantity of interest (estimand): β_1, as before.

What gets manipulated: the model. The population is unchanged, but we leave political interest out, as a survey without an interest question would have to:

$$\widehat{\text{Knowledge}}_i = \hat{\beta}_0^* + \hat{\beta}_1^* \text{Educ}_i$$

The confounder now sits in the error term, and it is related to education: assumption 2 fails. The lecture's omitted variable bias formula predicts the result, with $\hat{\delta}$ the slope from regressing the omitted variable on the included one:

$$E\left(\hat{\beta}_1^*\right) = \beta_1 + \beta_2 \delta, \quad \text{Interest}_i = \delta_0 + \delta \text{Educ}_i + u_i$$

```
ovb_delta <- population |>
  lm(interest ~ education, data = _) |>
  tidy() |>
  filter(term == "education") |>
  pull(estimate)
ovb_expected <- beta_education + beta_interest * ovb_delta
ovb_expected
```

OUTPUT

```
[1] 3.612298
```

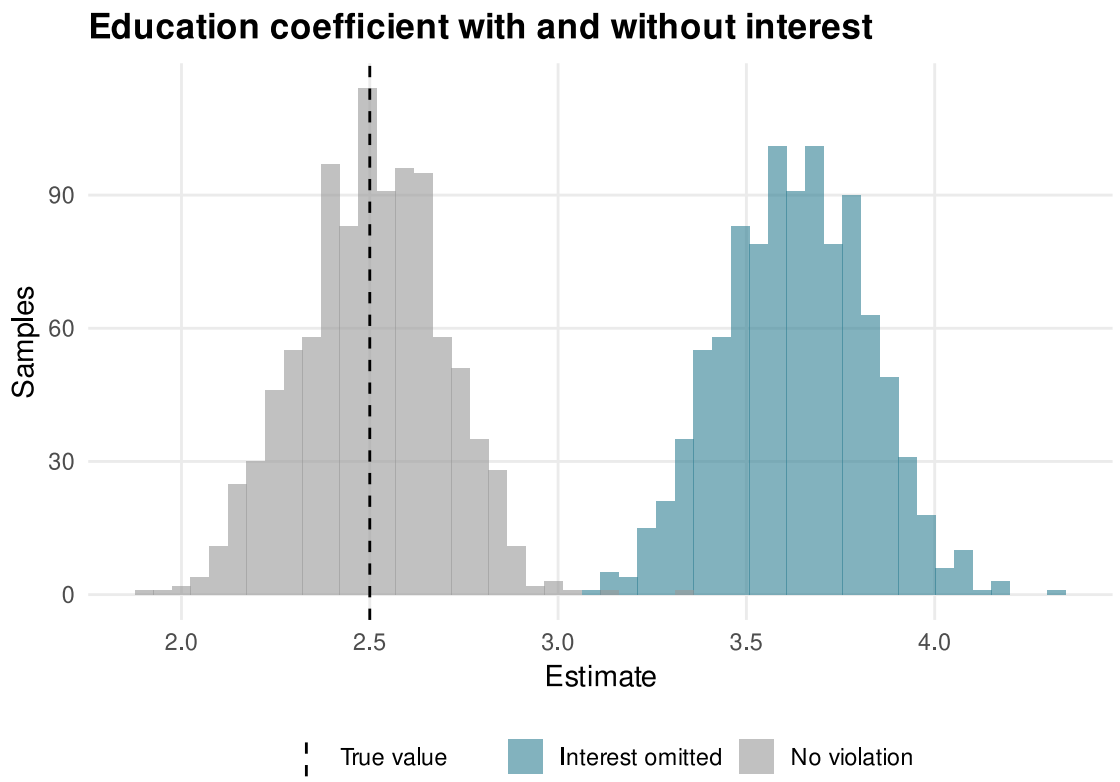
```
omitted <- mc_samples |>
  simulate_round(knowledge ~ education, "Interest omitted")

omitted_summary <- omitted |>
  summarize_round(truth)
omitted_summary
```

OUTPUT

```
# A tibble: 1 × 7
  round            term      true_value mean_estimate  bias sd_estimate mean_se
  <chr>            <chr>          <dbl>         <dbl> <dbl>       <dbl>   <dbl>
1 Interest omitted education        2.5          3.63  1.13       0.191   0.201
```

```
omitted |>
  bind_rows(baseline) |>
  plot_rounds(truth, "Education coefficient with and without interest")
```




```
omitted_mean <- omitted_summary |>
  pull(mean_estimate)
omitted_sd <- omitted_summary |>
  pull(sd_estimate)
omitted_se <- omitted_summary |>
  pull(mean_se)
```

The whole sampling distribution shifts: on average the education coefficient is 3.63 instead of 2.5, close to the formula's 3.61. Education gets credit for what interest does. The standard error is still right for this wrong target (0.201 reported, 0.191 actual), so the confidence intervals are narrow around the wrong value. More respondents would not help: the bias stays, only the spread shrinks.

Measurement error

Quantity of interest (estimand): β_1 and β_2.

What gets manipulated: a predictor. Real surveys don't observe political interest, they ask about it. The answer is the true interest plus noise u_i, rounded to whole points on the 0–10 scale:

$$\text{Interest}_i^* = \text{Interest}_i + u_i, \quad u_i \sim \mathcal{N}(0, \sigma_u^2)$$

We add it to the population as `interest_reported`, and join the surveys again:

```
sigma_report <- 1.5

population <- population |>
  mutate(
    interest_reported = round(interest + rnorm(n(), 0, sigma_report)),
    interest_reported = pmin(pmax(interest_reported, interest_min), interest_max)
  )

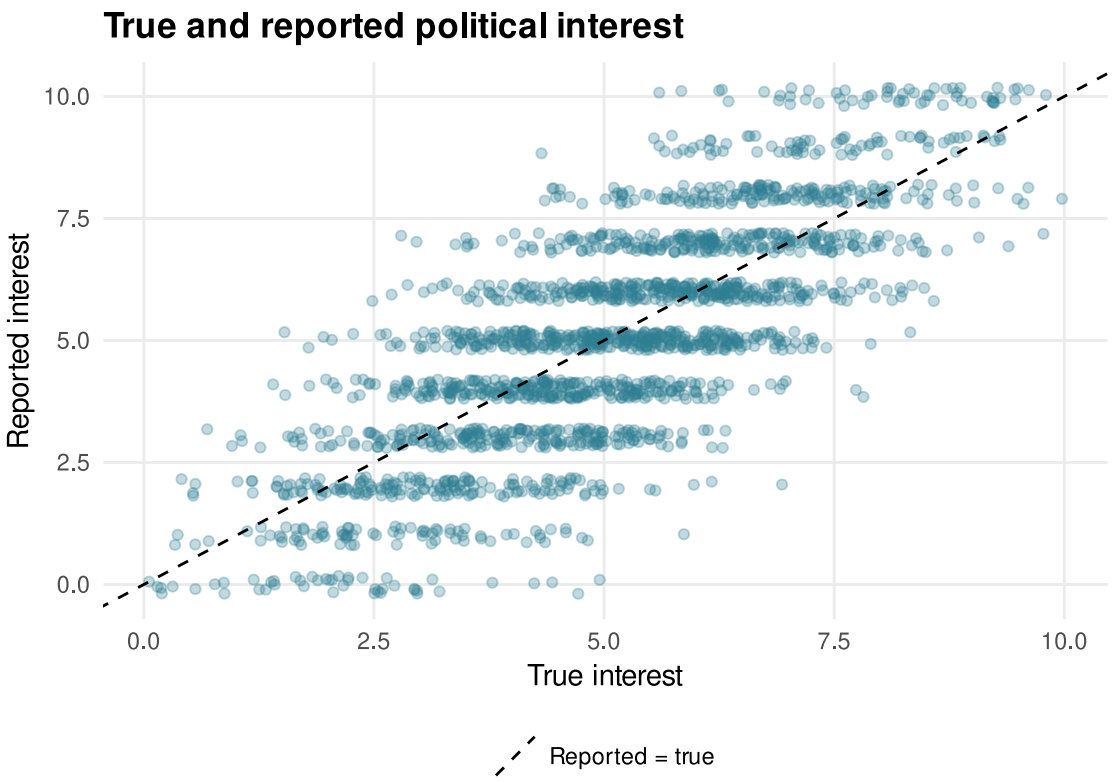
mc_samples <- mc_draws |>
  left_join(population, by = "voter_id")
```

The model uses the self-report, with $\sigma_u = 1.5$, instead of the true interest:

$$\widehat{\text{Knowledge}}_i = \hat{\beta}_0 + \hat{\beta}_1 \text{Educ}_i + \hat{\beta}_2 \text{Interest}^*_i$$

A first look at what the survey gets:

```r
population |>
  slice_sample(n = 2000) |>
  ggplot(aes(x = interest, y = interest_reported)) +
  geom_jitter(height = 0.2, alpha = 0.3, color = course_primary) +
  geom_abline(aes(intercept = 0, slope = 1, linetype = "Reported = true")) +
  scale_linetype_manual(values = c("Reported = true" = "dashed")) +
  labs(
    title    = "True and reported political interest",
    x        = "True interest",
    y        = "Reported interest",
    linetype = NULL
  ) +
  theme_intror()
```



How much of the reported variable's variance is true signal is its **reliability** λ. With education in the model as well, the expected interest coefficient shrinks by more than λ, because education already explains part of the reported interest (R^2 from regressing Interest* on education):

$$\lambda = \frac{\text{Var}(\text{Interest})}{\text{Var}(\text{Interest}^*)}, \quad E\big(\hat{\beta}_2\big) \approx \beta_2 \frac{\lambda - R^2}{1 - R^2}$$

```r
reliability <- population |>
  summarize(var(interest) / var(interest_reported)) |>
  pull()
reported_r2 <- population |>
  lm(interest_reported ~ education, data = _) |>
  glance() |>
  pull(r.squared)
attenuation_expected <- beta_interest * (reliability - reported_r2) / (1 - reported_r2)
attenuation_expected
```

OUTPUT

```
[1] 1.617472
```

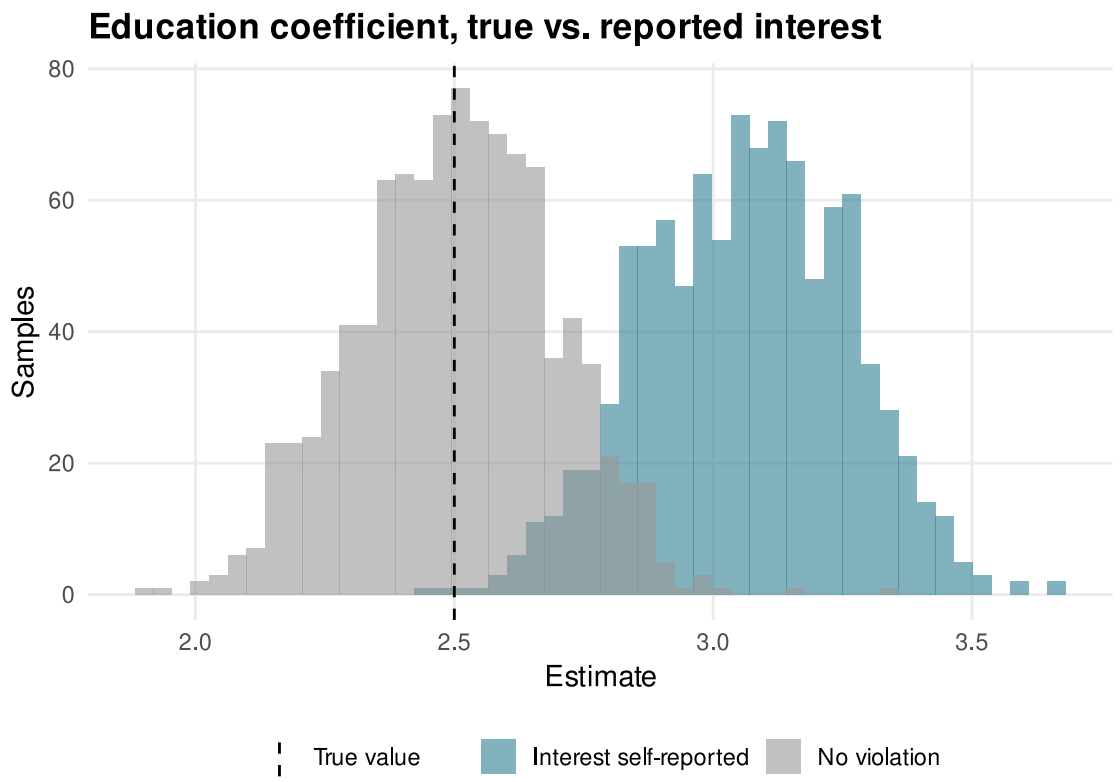
```r
measurement <- mc_samples |>
  simulate_round(knowledge ~ education + interest_reported, "Interest self-reported")

measurement_summary <- measurement |>
  summarize_round(truth)
measurement_summary
```

OUTPUT

```
# A tibble: 2 × 7
  round                term  true_value mean_estimate   bias sd_estimate mean_se
  <chr>                <chr>      <dbl>         <dbl>  <dbl>       <dbl>   <dbl>
1 Interest self-repor… educ…        2.5          3.06  0.562       0.195   0.198
2 Interest self-repor… inte…        3            1.58 -1.42        0.185   0.181
```

```
measurement |>
  bind_rows(baseline) |>
  plot_rounds(truth, "Education coefficient, true vs. reported interest")
```




```
measurement_education <- measurement_summary |>
  filter(term == "education") |>
  pull(mean_estimate)
measurement_interest <- measurement_summary |>
  filter(term == "interest") |>
  pull(mean_estimate)
```

Both coefficients are biased. The interest coefficient is **attenuated**, from 3 to 1.58 on average, near the formula's 1.62. The formula assumes noise that is neither rounded nor capped; our survey answers are both, so it is only approximate. The noise makes interest look less related to knowledge than it is. Education, measured without error, is biased too, upwards to 3.06: the noisy interest variable controls for the confounder only partly, so part of the omitted variable bias comes back. Measurement error in a control variable spills over to the coefficient we care about.

Heteroskedasticity

Quantity of interest (estimand): β_1.

What gets manipulated: the error term. A new outcome, `knowledge_hetero`, follows the same model, but the spread of the error grows with education. Among voters with little education, knowledge is similarly low. Among the highly educated, it differs a lot, depending on field and job. How fast the spread grows:

```
hetero_kappa <- 0.12
hetero_shift <- 7
```

$$\text{Knowledge}_i^{\text{hetero}} = \beta_0 + \beta_1 \text{Educ}_i + \beta_2 \text{Interest}_i + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_i^2), \quad \sigma_i = 0.12(\text{Educ}_i - 7)^2$$

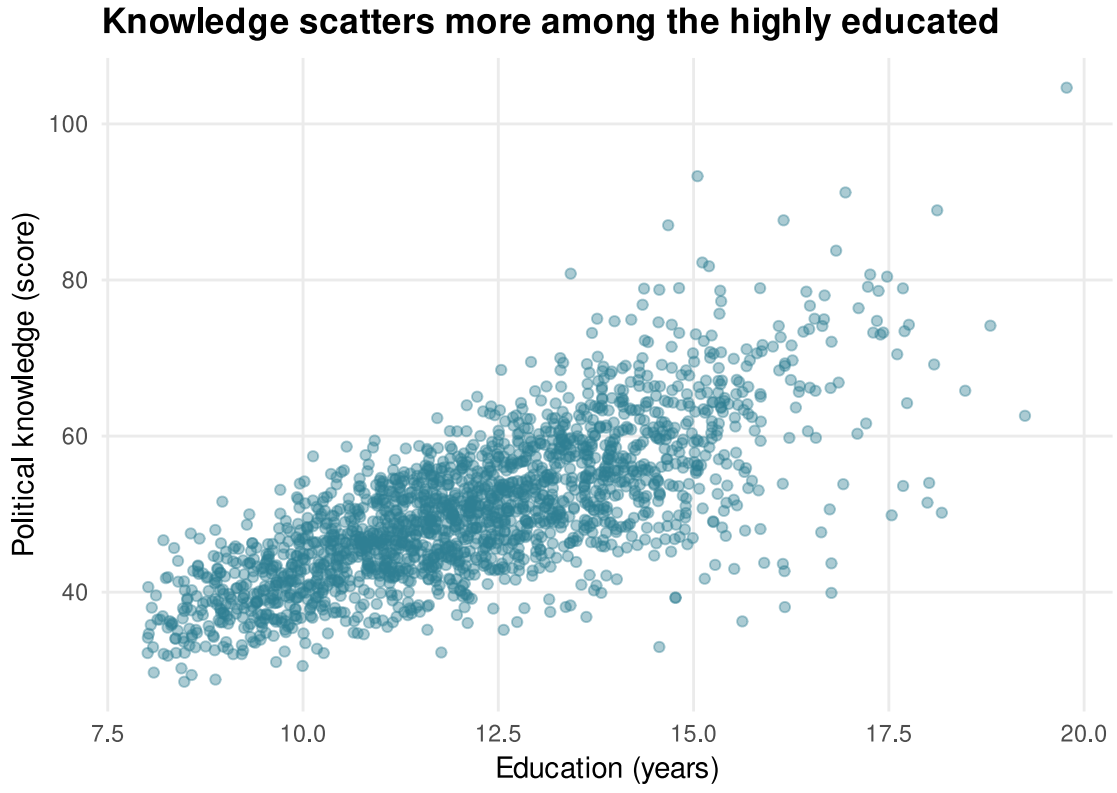
```
population <- population |>
  mutate(
    knowledge_hetero = beta_0 + beta_education * education + beta_interest * interest +
      rnorm(n(), 0, hetero_kappa * (education - hetero_shift)^2)
  )

mc_samples <- mc_draws |>
  left_join(population, by = "voter_id")
```

The model and the predictors stay as in the baseline:

$$\widehat{\text{Knowledge}}_i^{\text{hetero}} = \hat{\beta}_0 + \hat{\beta}_1 \text{Educ}_i + \hat{\beta}_2 \text{Interest}_i$$

```
population |>
  slice_sample(n = 2000) |>
  ggplot(aes(x = education, y = knowledge_hetero)) +
  geom_point(alpha = 0.4, color = course_primary) +
  labs(
    title = "Knowledge scatters more among the highly educated",
    x     = "Education (years)",
    y     = "Political knowledge (score)"
  ) +
  theme_intror()
```



Assumption 3 fails. The OLS standard error formula assumes one error variance σ_ε^2 for everyone, estimated by $\hat{\sigma}^2$ (with R_1^2 from regressing education on the other predictor):

$$\mathrm{SE}\left(\hat{\beta}_1\right) = \frac{\hat{\sigma}}{\sqrt{\sum_{i=1}^{n}\left(\mathrm{Educ}_i - \overline{\mathrm{Educ}}\right)^2 (1 - R_1^2)}}$$

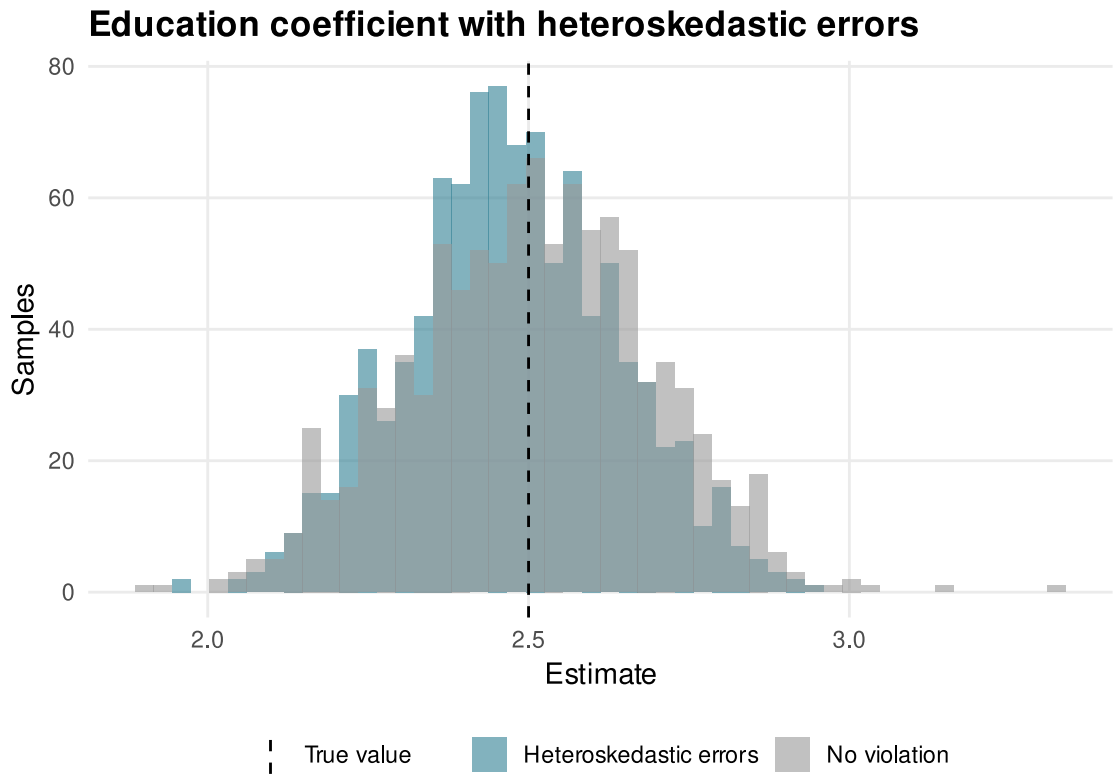
```
hetero <- mc_samples |>
  simulate_round(knowledge_hetero ~ education + interest, "Heteroskedastic errors")

hetero_summary <- hetero |>
  summarize_round(truth)
hetero_summary
```

OUTPUT

```
# A tibble: 2 × 7
  round              term  true_value mean_estimate     bias sd_estimate mean_se
  <chr>              <chr>      <dbl>         <dbl>    <dbl>       <dbl>   <dbl>
1 Heteroskedastic e… educ…        2.5          2.47 -0.0286        0.165   0.111
2 Heteroskedastic e… inte…        3            3.01  0.00892       0.134   0.133
```

```
hetero |>
  bind_rows(baseline) |>
  plot_rounds(truth, "Education coefficient with heteroskedastic errors")
```



```r
hetero_education <- hetero_summary |>
  filter(term == "education")
hetero_mean <- hetero_education |>
  pull(mean_estimate)
hetero_sd <- hetero_education |>
  pull(sd_estimate)
hetero_se <- hetero_education |>
  pull(mean_se)
```

The estimates are still centered on the truth (mean 2.47): heteroskedasticity does not bias $\hat{\beta}$. But they scatter with a standard deviation of 0.165, while `lm()` reports 0.111 on average, 32% too little. The confidence intervals are too narrow, and we would reject H_0 too often.

The bootstrap as a fix

The bootstrap from the quantities of interest above does not assume a common error variance: it resamples the respondents, each with their own error. Within the simulation, that means a bootstrap in every survey: resample the survey `n_boot_inner` times, refit, and take the standard deviation of the coefficients, with `infer` as in Week 7. That is $200 \times 200 = 40{,}000$ regressions, so we bootstrap only the first 200 surveys:

$$\mathrm{SE}_{\mathrm{boot}\left(\hat{\beta}_1\right)} = \mathrm{sd}\left(\tilde{\beta}_1^{(1)}, \ldots, \tilde{\beta}_1^{(B)}\right)$$

```r
n_sims_boot  <- 200
n_boot_inner <- 200
```

```r
hetero_boot_se <- mc_samples |>
  distinct(sample = sample_id) |>
  filter(sample <= n_sims_boot) |>
  rowwise() |>
  mutate(
    mc_samples |>
      filter(sample_id == sample) |>
      specify(knowledge_hetero ~ education + interest) |>
      generate(reps = n_boot_inner, type = "bootstrap") |>
      fit() |>
      ungroup() |>
      filter(term == "education") |>
      summarize(se_boot = sd(estimate))
  ) |>
  ungroup() |>
  rename(sample_id = sample)
hetero_boot_se
```

OUTPUT

```
# A tibble: 200 × 2
   sample_id se_boot
       <int>   <dbl>
 1         1   0.136
 2         2   0.157
 3         3   0.170
 4         4   0.219
 5         5   0.161
 6         6   0.155
 7         7   0.146
 8         8   0.181
 9         9   0.132
10        10   0.144
# ℹ 190 more rows
```

The three numbers side by side, for the same 200 surveys:

```r
hetero_se_compare <- hetero |>
  filter(sample_id <= n_sims_boot, term == "education") |>
  left_join(hetero_boot_se, by = "sample_id") |>
  summarize(
    sd_estimate  = sd(estimate),
    mean_se_ols  = mean(std.error),
    mean_se_boot = mean(se_boot)
  )
hetero_se_compare
```

OUTPUT

```
# A tibble: 1 × 3
  sd_estimate mean_se_ols mean_se_boot
        <dbl>       <dbl>        <dbl>
1       0.162       0.112        0.159
```

```
hetero_boot_mean <- hetero_se_compare |>
  pull(mean_se_boot)
hetero_ols_mean <- hetero_se_compare |>
  pull(mean_se_ols)
hetero_actual_sd <- hetero_se_compare |>
  pull(sd_estimate)
```

The bootstrap standard error (0.159 on average) is close to the actual spread (0.162); the OLS one (0.112) is not. This is what Week 7's counties showed: the bootstrap standard errors came out larger than the OLS ones because the counties scatter unequally around the line.

Robust standard errors

The analytical fix is a **heteroskedasticity-consistent** ("robust") standard error, which estimates each observation's error variance from its own residual. `marginaleffects` computes it with `vcov = "HC3"`:

```
mc_samples |>
  filter(sample_id == 1) |>
  lm(knowledge_hetero ~ education + interest, data = _) |>
  hypotheses(vcov = "HC3")
```

Multicollinearity

Quantity of interest (estimand): β_1.

What gets manipulated: the relationship between the predictors. Imagine a second society, in which schooling depends almost entirely on early political interest. Its education, `education_collinear`, has the same mean and spread as `education`, but is nearly determined by interest. It is its own population, built from the same voters' interest, and again nobody has fewer than 8 years:

```
delta_0_collinear  <- 6
delta_collinear    <- 1.2
sigma_nu_collinear <- 0.6
```

$$\text{Educ}_i^{\text{col}} = 6 + 1.2\text{Interest}_i + \nu_i^{\text{col}}, \quad \nu_i^{\text{col}} \sim \mathcal{N}(0, 0.6^2), \quad \text{Educ}_i^{\text{col}} \geq 8$$

Knowledge in this society, `knowledge_collinear`, follows the true model with the same βs:

$$\text{Knowledge}_i^{\text{col}} = \beta_0 + \beta_1 \text{Educ}_i^{\text{col}} + \beta_2 \text{Interest}_i + \varepsilon_i$$

```
population_collinear <- population |>
  mutate(
    education_collinear = delta_0_collinear + delta_collinear * interest + rnorm(n(),
0, sigma_nu_collinear),
    knowledge_collinear = beta_0 + beta_education * education_collinear + beta_interest
* interest +
      rnorm(n(), 0, sigma_epsilon)
  ) |>
  filter(education_collinear >= education_min)
```

A different population needs its own surveys, drawn as in step 2:

```
mc_samples_collinear <- population_collinear |>
  slice_sample(n = n_sample * n_sims, replace = TRUE) |>
  mutate(sample_id = rep(1:n_sims, each = n_sample))
```

The model we estimate:

$$\widehat{\text{Knowledge}}_i^{\text{col}} = \hat{\beta}_0 + \hat{\beta}_1 \text{Educ}_i^{\text{col}} + \hat{\beta}_2 \text{Interest}_i$$

Both predictors side by side, in the original population and in the second society, 2,000 voters each, with the OLS line and the R^2 of regressing education on interest:

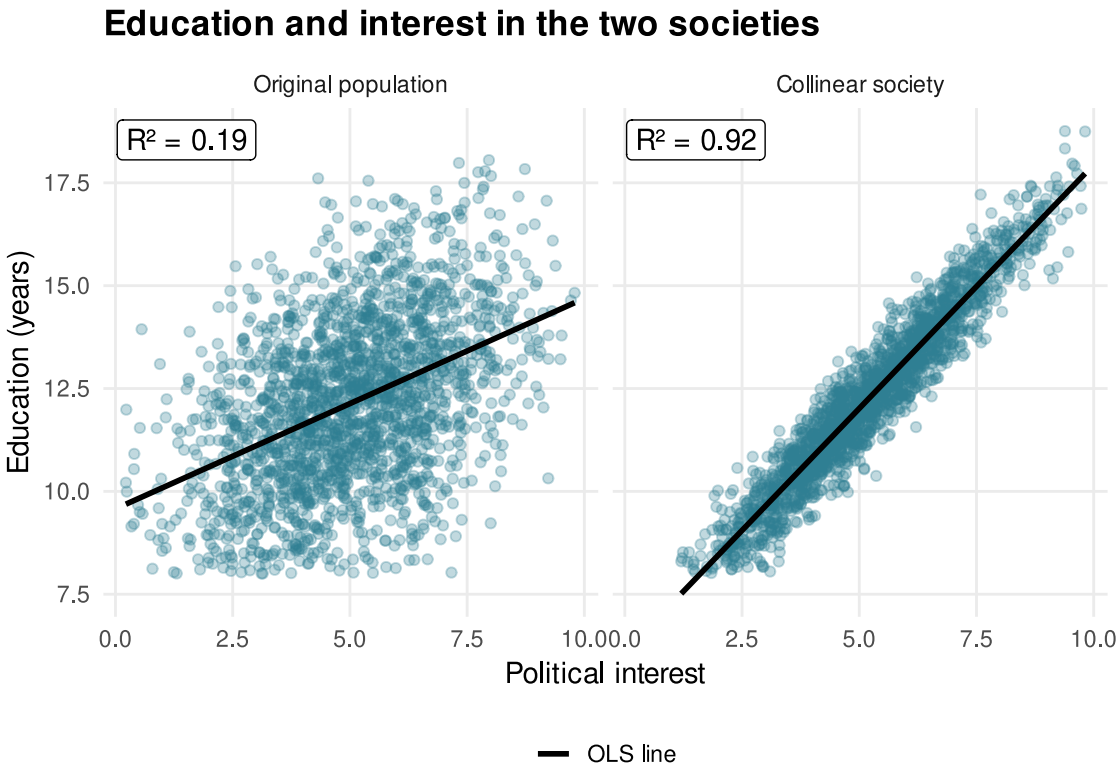
```
original_pairs <- population |>
  slice_sample(n = 2000) |>
  select(interest, years = education) |>
  mutate(society = "Original population")
collinear_pairs <- population_collinear |>
  slice_sample(n = 2000) |>
  select(interest, years = education_collinear) |>
  mutate(society = "Collinear society")
predictor_pairs <- bind_rows(original_pairs, collinear_pairs)

predictor_r2 <- predictor_pairs |>
  group_by(society) |>
  summarize(r2 = cor(years, interest)^2) |>
  mutate(label = str_glue("R² = {format(round(r2, 2), nsmall = 2)}"))
predictor_r2
```

OUTPUT

```
# A tibble: 2 × 3
  society                r2 label
  <chr>               <dbl> <glue>
1 Collinear society   0.917 R² = 0.92
2 Original population 0.191 R² = 0.19
```

```r
predictor_pairs |>
  ggplot(aes(x = interest, y = years)) +
  geom_point(alpha = 0.3, color = course_primary) +
  geom_smooth(aes(linetype = "OLS line"), method = "lm", formula = y ~ x, se = FALSE,
color = "black") +
  geom_label(data = predictor_r2, aes(x = -Inf, y = Inf, label = label), hjust = -0.1,
vjust = 1.3) +
  facet_wrap(~ fct_inorder(society)) +
  scale_linetype_manual(values = c("OLS line" = "solid")) +
  labs(
    title    = "Education and interest in the two societies",
    x        = "Political interest",
    y        = "Education (years)",
    linetype = NULL
  ) +
  theme_intror()
```



Nothing is biased here: no assumption fails. What changes is how much education varies *on its own*. The standard error formula from the heteroskedasticity section shows why: it divides by $1 - R_1^2$, where R_1^2 is how well interest predicts education, the R^2 in the plot. The **variance inflation factor** says how much larger the variance of $\hat{\beta}_1$ becomes because of it:

$$\text{VIF}_1 = \frac{1}{1 - R_1^2}$$

```r
vif_values <- predictor_r2 |>
  mutate(vif = 1 / (1 - r2))
vif_values
```

OUTPUT

```
# A tibble: 2 × 4
  society                r2 label       vif
  <chr>               <dbl> <glue>    <dbl>
1 Collinear society   0.917 R² = 0.92 12.1
2 Original population 0.191 R² = 0.19  1.24
```

Education has almost the same spread in both societies, so the standard error should grow by the square root of the ratio of the two VIFs:

$$\frac{\mathrm{SE}\left(\hat{\beta}_1^{\mathrm{col}}\right)}{\mathrm{SE}\left(\hat{\beta}_1\right)} \approx \sqrt{\frac{\mathrm{VIF}_1^{\mathrm{col}}}{\mathrm{VIF}_1}}$$

```r
vif_original <- vif_values |>
  filter(society == "Original population") |>
  pull(vif)
vif_collinear <- vif_values |>
  filter(society == "Collinear society") |>
  pull(vif)
se_ratio_expected <- sqrt(vif_collinear / vif_original)
se_ratio_expected
```

OUTPUT

```
[1] 3.124378
```

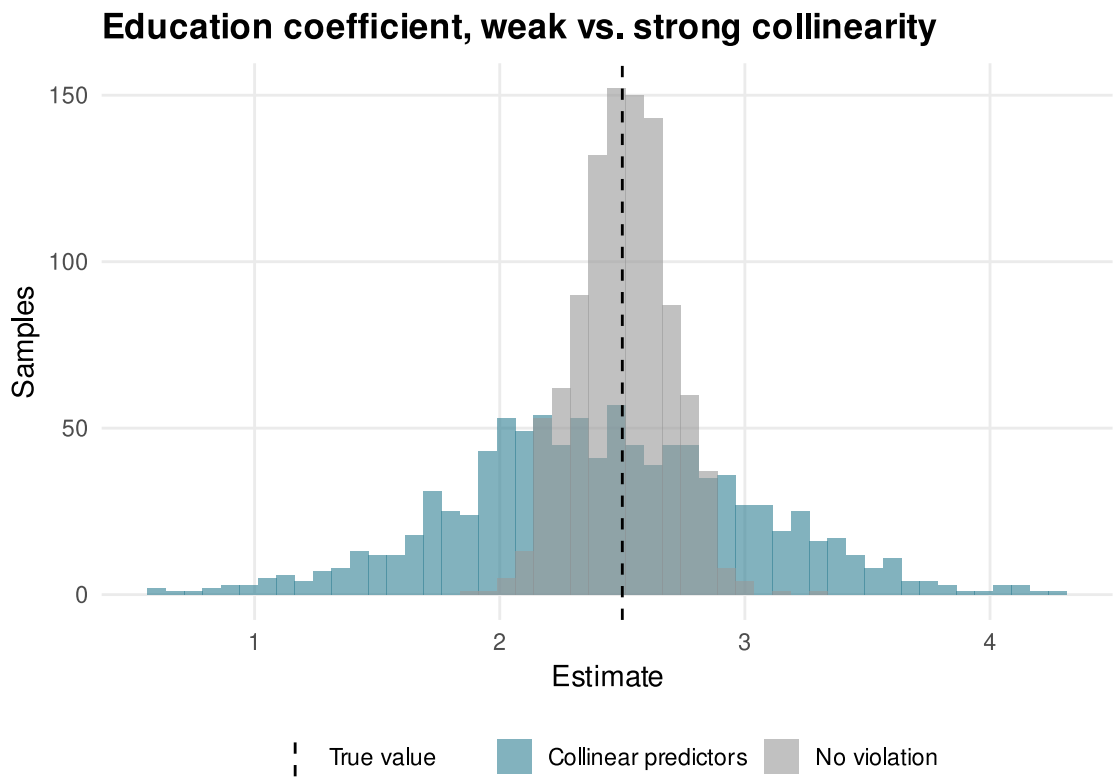
```r
collinear <- mc_samples_collinear |>
   simulate_round(knowledge_collinear ~ education_collinear + interest, "Collinear
predictors")

collinear_summary <- collinear |>
  summarize_round(truth)
collinear_summary
```

OUTPUT

```
# A tibble: 2 × 7
  round               term  true_value mean_estimate    bias sd_estimate mean_se
  <chr>               <chr>      <dbl>         <dbl>   <dbl>       <dbl>   <dbl>
1 Collinear predicto… educ…        2.5          2.43 -0.0748       0.602   0.602
2 Collinear predicto… inte…        3            3.09  0.0916       0.737   0.746
```

```
collinear |>
  bind_rows(baseline) |>
  plot_rounds(truth, "Education coefficient, weak vs. strong collinearity")
```




```
collinear_education <- collinear_summary |>
  filter(term == "education")
collinear_mean <- collinear_education |>
  pull(mean_estimate)
collinear_sd <- collinear_education |>
  pull(sd_estimate)
collinear_se <- collinear_education |>
  pull(mean_se)
```

The estimates are still centered on the truth (2.43 on average), but they are much wider: the standard deviation of the education estimates grows from 0.189 to 0.602, a factor of 3.2, as the VIFs predicted (3.1). Unlike under heteroskedasticity, the reported standard error knows this (0.602 on average): OLS is honest about how little the data can separate education from interest. Dropping interest to get a smaller standard error would bring back the omitted variable bias.

All rounds compared

The education coefficient in every round, with the middle 95% of its sampling distribution:

```r
all_rounds <- bind_rows(baseline, omitted, measurement, hetero, collinear) |>
  mutate(round = fct_inorder(round))

all_rounds_summary <- all_rounds |>
  summarize_round(truth) |>
  filter(term == "education") |>
  mutate(se_ratio = mean_se / sd_estimate)
all_rounds_summary
```

OUTPUT

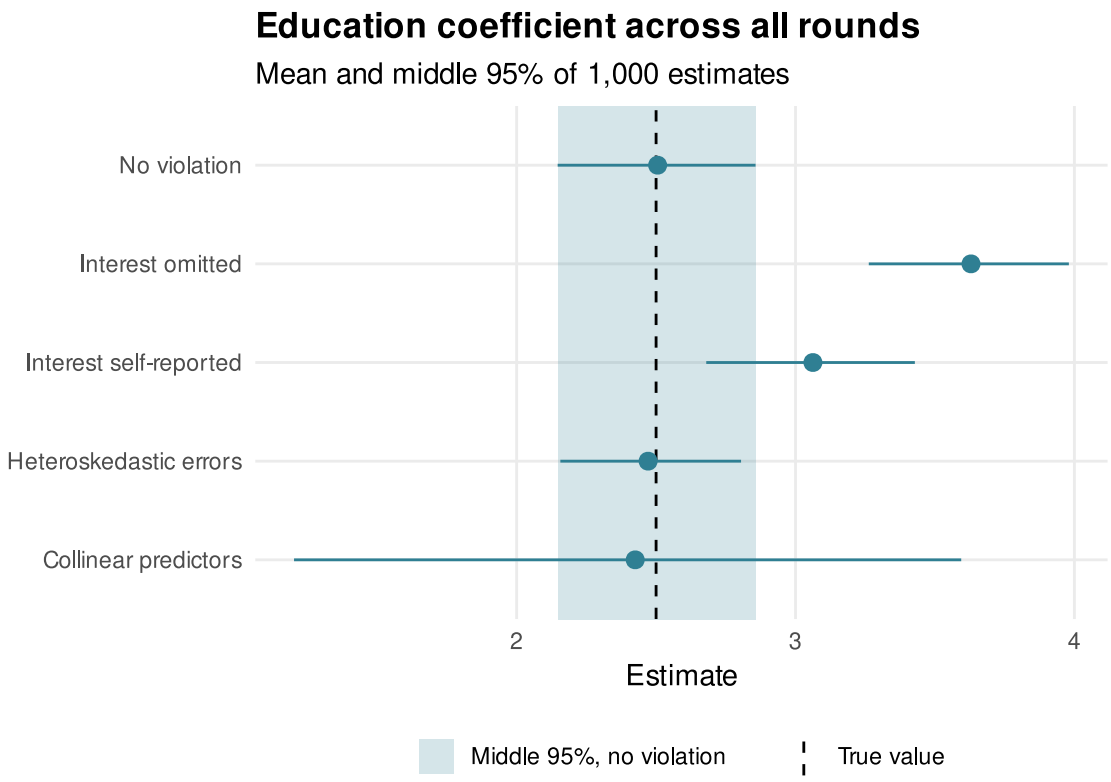
```
# A tibble: 5 × 8
  round     term  true_value mean_estimate     bias sd_estimate mean_se se_ratio
  <fct>     <chr>      <dbl>         <dbl>    <dbl>       <dbl>   <dbl>    <dbl>
1 No viola… educ…        2.5          2.51  0.00551       0.189   0.193    1.02
2 Interest… educ…        2.5          3.63  1.13          0.191   0.201    1.05
3 Interest… educ…        2.5          3.06  0.562         0.195   0.198    1.02
4 Heterosk… educ…        2.5          2.47 -0.0286        0.165   0.111    0.676
5 Collinea… educ…        2.5          2.43 -0.0748        0.602   0.602    0.999
```

The plot gets a function, since the Try-its below add rounds to it. The shaded band marks the middle 95% of the no-violation round, as a reference for the other rounds' intervals:

```r
plot_all_rounds <- function(results, truth) {
  round_intervals <- results |>
    filter(term == "education") |>
    group_by(round = fct_inorder(round)) |>
    summarize(
      mean_estimate = mean(estimate),
      lower         = quantile(estimate, 0.025),
      upper         = quantile(estimate, 0.975)
    )
  benchmark_interval <- round_intervals |>
    filter(round == "No violation")
  true_education <- truth |>
    filter(term == "education")

  round_intervals |>
    ggplot(aes(x = mean_estimate, y = fct_rev(round))) +
    geom_rect(
      data        = benchmark_interval,
       aes(xmin = lower, xmax = upper, ymin = -Inf, ymax = Inf, fill = "Middle 95%,
no violation"),
      inherit.aes = FALSE,
      alpha       = 0.2
    ) +
     geom_vline(data = true_education, aes(xintercept = true_value, linetype = "True
value")) +
    geom_pointrange(aes(xmin = lower, xmax = upper), color = course_primary) +
    scale_fill_manual(values = c("Middle 95%, no violation" = course_primary)) +
    scale_linetype_manual(values = c("True value" = "dashed")) +
    labs(
      title    = "Education coefficient across all rounds",
      subtitle = "Mean and middle 95% of 1,000 estimates",
      x        = "Estimate",
      y        = NULL,
      fill     = NULL,
      linetype = NULL
    ) +
    theme_intror()
}
```

```
all_rounds |>
  plot_all_rounds(truth)
```



Round	What we changed	Point estimate	Standard error
No violation	nothing	unbiased	right
Omitted variable	the model: a confounder left out	biased	right, around the wrong value
Measurement error	a predictor: the noisy self-report	biased (both coefficients)	right, around the wrong value
Heteroskedasticity	the error term: its spread grows with education	unbiased	too small; the bootstrap fixes it
Multicollinearity	the predictors: nearly determined by each other	unbiased	large, and reported correctly

Concluding remarks

Quantities of interest turn coefficients into statements in the units of the outcome, and the bootstrap gives each its uncertainty with the same five steps. A Monte Carlo simulation reverses the situation: we set the truth and watch what the estimator does with it. Violations of the zero conditional mean assumption, such as an omitted confounder or a noisy control, bias the estimate, and no sample size fixes that. Heteroskedasticity leaves the estimate unbiased but makes the reported standard error wrong. Multicollinearity makes the estimate imprecise, but OLS reports that honestly.

Try it: survey size

Last week's quantities of interest grew more certain with more data. The same should hold for the coefficients. Re-run the no-violation round with surveys of 2,000 instead of 500 voters.

Your task: report the standard deviation of the education estimates and the mean reported standard error with 2,000 respondents per survey, and how much smaller both are than with 500.

```
n_sample_large <- 2000

mc_samples_large <- population |>
  slice_sample(n = n_sample_large * n_sims, replace = TRUE) |>
  mutate(sample_id = rep(1:n_sims, each = n_sample_large), .before = 1)

mc_samples_large |>
  simulate_round(knowledge ~ education + interest, "No violation, n = 2000") |>
  bind_rows(baseline) |>
  summarize_round(truth) |>
  filter(term == "education")
```

OUTPUT

```
# A tibble: 2 × 7
  round              term  true_value mean_estimate    bias sd_estimate mean_se
  <chr>              <chr>      <dbl>         <dbl>   <dbl>       <dbl>   <dbl>
1 No violation       educ…        2.5          2.51 0.00551      0.189   0.193
2 No violation, n = … educ…        2.5          2.51 0.00775      0.0978  0.0965
```

Try it: stronger measurement error

Some survey questions are noisier than others. Suppose respondents report their interest much less precisely, with $\sigma_u = 3$ instead of 1.5.

Your task: report the mean estimates of both coefficients with this noisier self-report, and compare their bias with the measurement-error round above.

```r
sigma_report_large <- 3

population_noisy <- population |>
  mutate(
    interest_reported = round(interest + rnorm(n(), 0, sigma_report_large)),
    interest_reported = pmin(pmax(interest_reported, interest_min), interest_max)
  )

mc_draws |>
  left_join(population_noisy, by = "voter_id") |>
   simulate_round(knowledge ~ education + interest_reported, "Interest self-reported,
noisier") |>
  bind_rows(measurement) |>
  summarize_round(truth)
```

OUTPUT

```
# A tibble: 4 × 7
  round                term  true_value mean_estimate   bias sd_estimate mean_se
  <chr>                <chr>      <dbl>         <dbl>  <dbl>       <dbl>   <dbl>
1 Interest self-repor… educ…        2.5          3.06   0.562       0.195   0.198
2 Interest self-repor… inte…        3            1.58  -1.42        0.185   0.181
3 Interest self-repor… educ…        2.5          3.40   0.897       0.194   0.201
4 Interest self-repor… inte…        3            0.713 -2.29        0.135   0.137
```

Try it: curved education effect

Assumption 1, linearity. A year at school may teach more about politics than a year at university: the effect of education flattens out. Suppose knowledge follows a curve, with $\gamma < 0$:

$$\text{Knowledge}_i^{\text{curved}} = \beta_0 + \beta_1 \text{Educ}_i + \gamma(\text{Educ}_i - 10)^2 + \beta_2 \text{Interest}_i + \varepsilon_i$$

The effect of one more year is $\beta_1 + 2\gamma(\text{Educ}_i - 10)$: it equals $\beta_1 = 2.5$ at 10 years of education and gets smaller above. Use $\gamma = -0.2$. In a formula, a squared term is written as `I((education - 10)^2)`.

Your task: estimate the effect of education at 10 years, once with the linear model and once with the squared term added. Report both mean estimates, and add both rounds to the plot of all rounds.

```r
education_reference <- 10
gamma_curve <- -0.2

population <- population |>
  mutate(
    knowledge_curved = beta_0 + beta_education * education + gamma_curve * (education
- education_reference)^2 +
      beta_interest * interest + rnorm(n(), 0, sigma_epsilon)
  )

mc_samples <- mc_draws |>
  left_join(population, by = "voter_id")

curved_linear <- mc_samples |>
 simulate_round(knowledge_curved ~ education + interest, "Curved effect: linear model")
curved_squared <- mc_samples |>
  simulate_round(
    knowledge_curved ~ education + I((education - education_reference)^2) + interest,
    "Curved effect: squared term"
  )

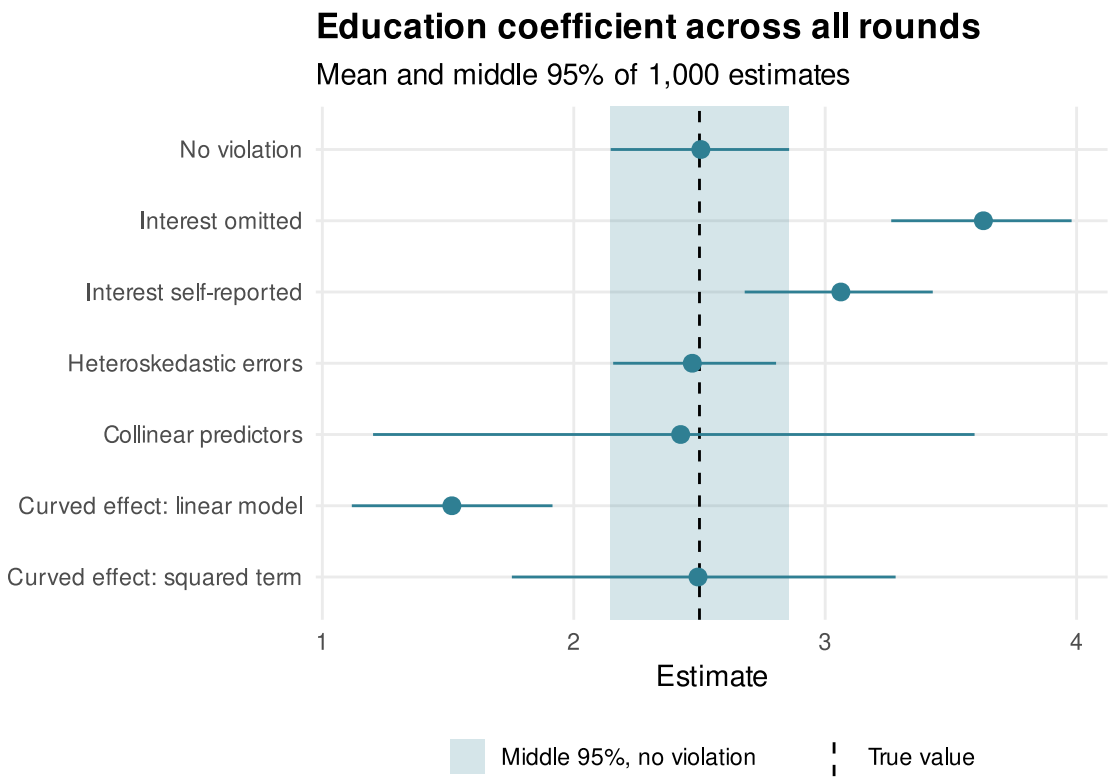
bind_rows(curved_linear, curved_squared) |>
  summarize_round(truth)
```

OUTPUT

```
# A tibble: 4 × 7
  round              term  true_value mean_estimate     bias sd_estimate mean_se
  <chr>              <chr>      <dbl>         <dbl>    <dbl>       <dbl>   <dbl>
1 Curved effect: li… educ…        2.5          1.52 -0.985         0.204   0.195
2 Curved effect: li… inte…        3            3.03  0.0348        0.237   0.232
3 Curved effect: sq… educ…        2.5          2.49 -0.00586       0.393   0.378
4 Curved effect: sq… inte…        3            3.03  0.0288        0.236   0.230
```

```r
all_rounds <- all_rounds |>
  bind_rows(curved_linear, curved_squared)

all_rounds |>
  plot_all_rounds(truth)
```



Try it: clustered surveys

Assumption 4, independence. Many surveys first draw municipalities, then voters within them. Voters in the same municipality share local schools and local news, so their education and their knowledge have a common local part. Suppose 2,000 municipalities, each with a local shift a_m in education and b_m in knowledge:

$$\text{Educ}_i^{\text{clust}} = \text{Educ}_i + a_{m[i]}, \quad a_m \sim \mathcal{N}(0, 1.5^2)$$

$$\text{Knowledge}_i^{\text{clust}} = \beta_0 + \beta_1 \text{Educ}_i^{\text{clust}} + \beta_2 \text{Interest}_i + b_{m[i]} + \varepsilon_i, \quad b_m \sim \mathcal{N}(0, 6^2)$$

Each survey draws 25 municipalities and 20 voters in each of them, again 500 respondents.

Your task: simulate 1,000 such surveys. Report the mean education estimate, the standard deviation of the estimates and the mean reported standard error, and add the round to the plot of all rounds.

```r
n_municipalities      <- 2000
sigma_education_local <- 1.5
sigma_knowledge_local <- 6
n_clusters            <- 25
n_per_cluster         <- 20

municipalities <- tibble(municipality = 1:n_municipalities) |>
  mutate(
    education_local = rnorm(n(), 0, sigma_education_local),
    knowledge_local = rnorm(n(), 0, sigma_knowledge_local)
  )

population_clustered <- population |>
  mutate(municipality = rep(1:n_municipalities, length.out = n())) |>
  left_join(municipalities, by = "municipality") |>
  mutate(
    education_clustered = education + education_local,
    knowledge_clustered = beta_0 + beta_education * education_clustered + beta_interest
* interest +
      knowledge_local + rnorm(n(), 0, sigma_epsilon)
  )

clustered <- expand_grid(sample_id = 1:n_sims, municipality = 1:n_municipalities) |>
  group_by(sample_id) |>
  slice_sample(n = n_clusters) |>
  ungroup() |>
  left_join(population_clustered, by = "municipality", relationship = "many-to-many")
|>
  group_by(sample_id, municipality) |>
  slice_sample(n = n_per_cluster) |>
  ungroup() |>
   simulate_round(knowledge_clustered ~ education_clustered + interest, "Clustered
surveys")

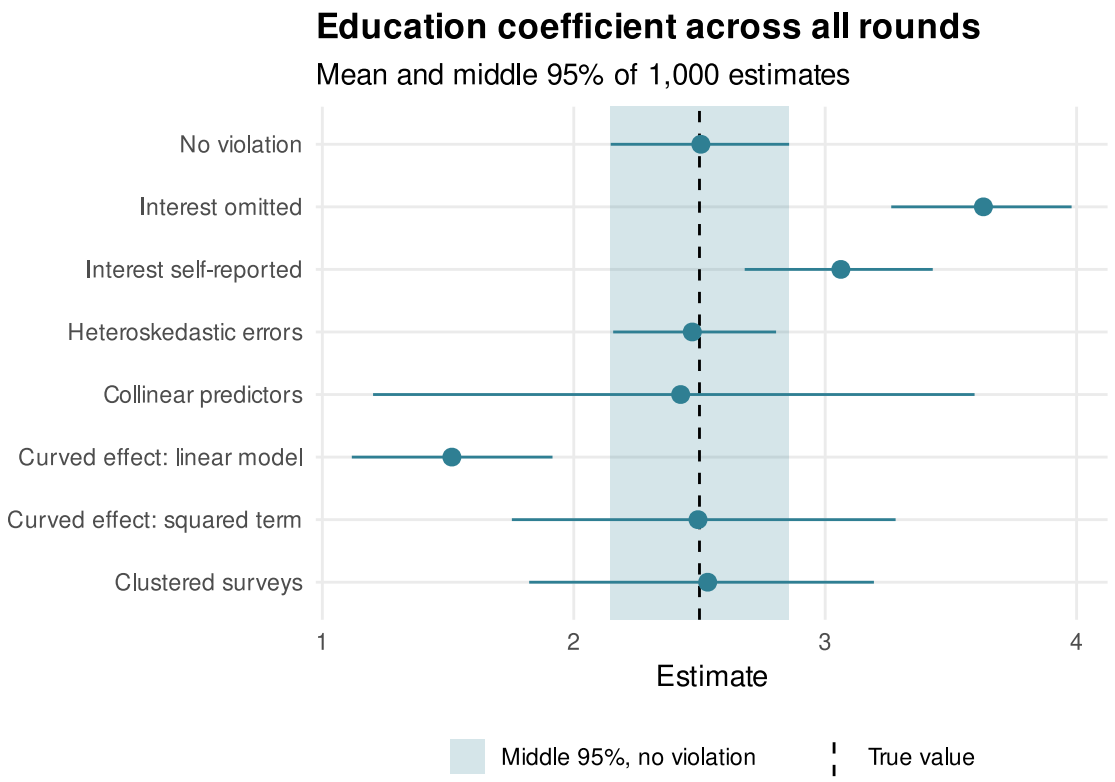
clustered |>
  summarize_round(truth)
```

OUTPUT

```
# A tibble: 2 × 7
  round             term    true_value mean_estimate    bias sd_estimate mean_se
  <chr>             <chr>        <dbl>         <dbl>   <dbl>       <dbl>   <dbl>
1 Clustered surveys educat…        2.5          2.53  0.0327       0.352   0.187
2 Clustered surveys intere…        3            2.97 -0.0340       0.312   0.272
```

```
all_rounds <- all_rounds |>
  bind_rows(clustered)

all_rounds |>
  plot_all_rounds(truth)
```



Try it: skewed errors

Assumption 6, normality. Errors need not be Normal: suppose most voters know a little less than the model predicts and a few know much more. A Gamma distribution from Week 2 gives such a skewed error, shifted and scaled to mean 0 and standard deviation σ_ε:

$$\varepsilon_i = \sigma_\varepsilon \frac{g_i - k}{\sqrt{k}}, \quad g_i \sim \text{Gamma}(k, 1), \quad k = 1$$

Your task: report the mean education estimate, the standard deviation of the estimates and the mean reported standard error with skewed errors, and add the round to the plot of all rounds.

```r
skew_shape <- 1

population <- population |>
  mutate(
    knowledge_skewed = beta_0 + beta_education * education + beta_interest * interest +
      sigma_epsilon * (rgamma(n(), shape = skew_shape) - skew_shape) / sqrt(skew_shape)
  )

mc_samples <- mc_draws |>
  left_join(population, by = "voter_id")

skewed <- mc_samples |>
  simulate_round(knowledge_skewed ~ education + interest, "Skewed errors")

skewed |>
  summarize_round(truth)
```

OUTPUT

```
# A tibble: 2 × 7
  round         term      true_value mean_estimate    bias sd_estimate mean_se
  <chr>         <chr>          <dbl>         <dbl>   <dbl>       <dbl>   <dbl>
1 Skewed errors education        2.5          2.47 -0.0268       0.195   0.193
2 Skewed errors interest         3            3.01  0.0116       0.241   0.229
```

```r
all_rounds <- all_rounds |>
  bind_rows(skewed)

all_rounds |>
  plot_all_rounds(truth)
```

